



Amazon CloudWatch

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Amazon CloudWatch



A collection of monitoring services for **logging**, **reacting**, **visualizing** log data



Introduction to CloudWatch

AWS CloudWatch is a **monitoring solution** for your AWS resources



CloudWatch is an **umbrella** service, meaning that it is really a **collection of monitoring tools** as follows:

Logs — any custom log data, Application Logs, Nginx Logs, Lambda Logs

Metrics Represents a time-ordered set of data points. A variable to monitor. eg. Memory Usage

Events — trigger an event based on a condition eg. every hour, take a snapshot of the server (now known as **Amazon EventBridge**)

Alarms — triggers notifications based on metrics that breach a defined threshold

Dashboards — create visualizations based on metrics

ServiceLens — visualize and analyze the health, performance, and availability of your app in a single place

Container Insights — collects, aggregates, and summarizes metrics and logs from your containerized apps and microservices

Synthetics — test your web apps to see if they're broken

Contributor Insights — view the top contributors impacting the performance of your systems and applications in real-time



All



CloudWatch



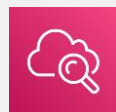
Services



Build Off Of

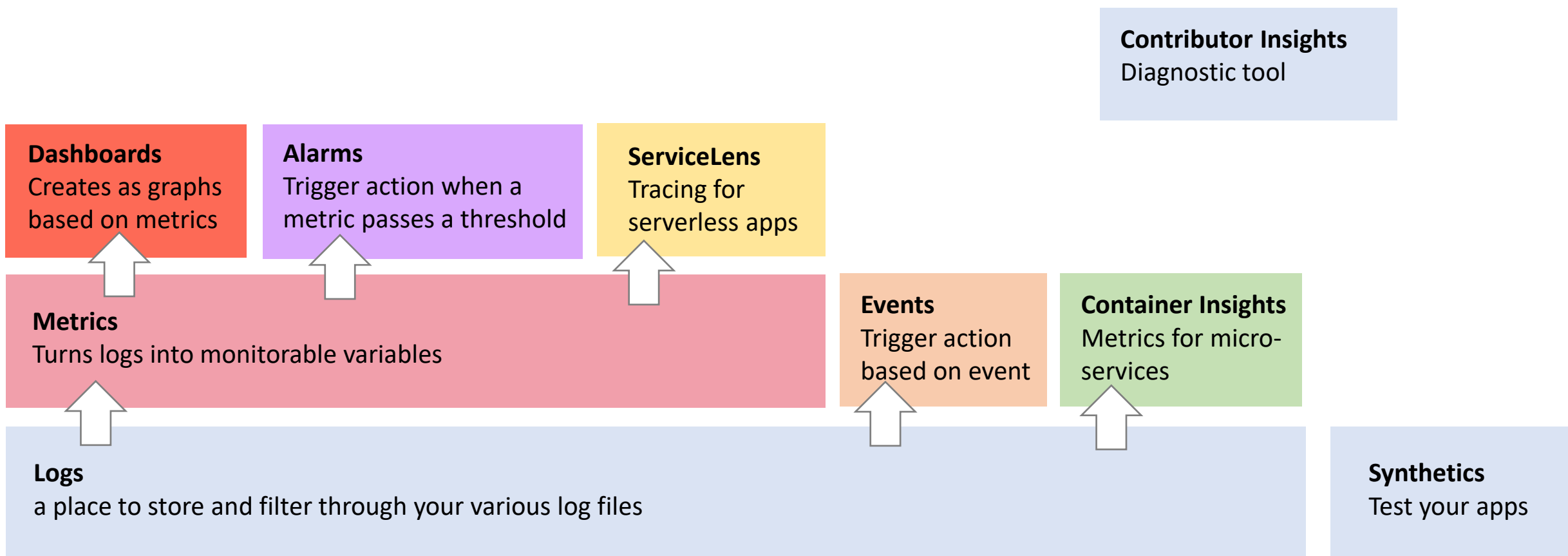


CloudWatch Logs!



Introduction to CloudWatch

CloudWatch Logs is the basis for many CloudWatch Services





The Pillars of Observability

What is Observability?

The ability to measure and understand how internal systems work in order to answer questions regarding performance, tolerance, Security and faults with a system/application.

To obtain observability, you need to use **Metrics**, **Logs**, and **Traces**.

You have to use them together. Using them in isolate does not gain you observability

Metrics

A number that is measured over a period of time
eg. If we measured the CPU usage and aggregated it over a period of time, we could have an **Average CPU metric**

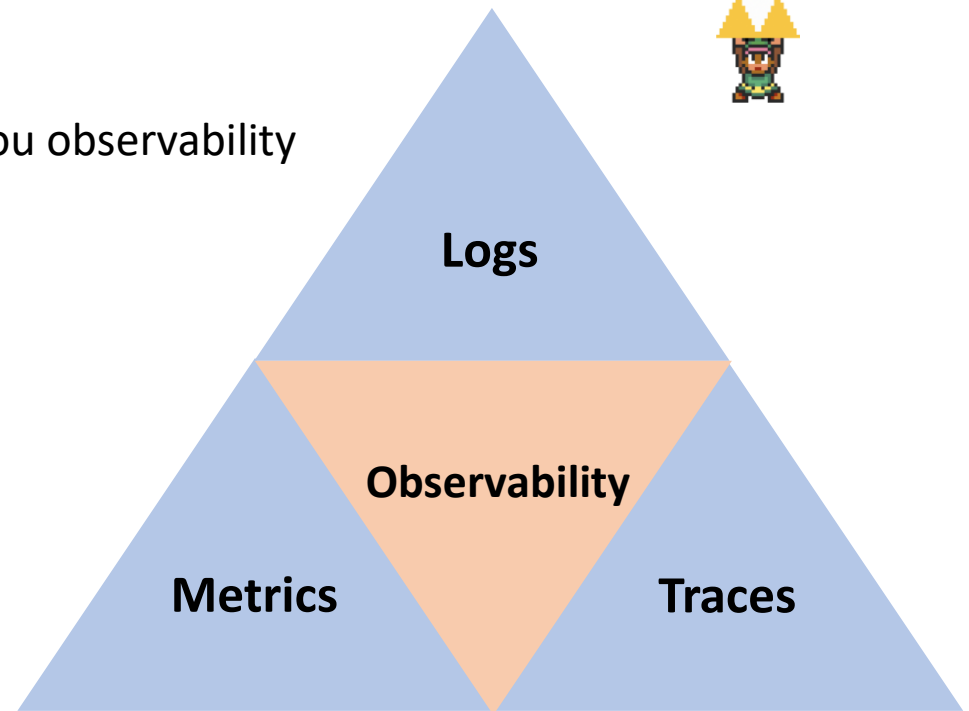
Logs

A text file where each line contains event data about what happened at a certain time.

Traces

A history of requests that travels through multiple Apps/services so we can pinpoint performance or failure.

Looks like they should have called it the **Triforce of Observability**



***Alarms** it sometimes considered the **fourth** Pillar of Observability.



CloudWatch Logs

CloudWatch Logs is used to **monitor**, **store**, and **access** your log files

CloudWatch is a **centralized log management service**.



Export Logs to S3

You can export Logs to S3 to do things like perform custom analysis



Stream to Elasticsearch Service (ES)

You can stream logs to an ES cluster in near real-time to have a more robust full-text search or use with the ELK stack



Stream CloudTrail Events to CloudWatch Logs

You can turn on CloudTrail to stream event data to a CloudWatch Log Group



Log Security

By default, log groups are encrypted at rest using SSE. You can use your own Customer Master Key (CMKs) with AWS KMS

Log Filtering

Logs can be filtered using a Filtering Syntax, and CloudWatch Logs has a sub-service called **CloudWatch Insights**

Log Retention

By default, logs are **kept indefinitely and never expire**. You can adjust the retention policy for each log group:

- keeping the indefinite retention
- choosing a retention period between 1 Day to 10 Years

Most AWS Services are integrated with CloudWatch Logs.

Logging of services **sometimes needs to be turned** on or requires the IAM Permissions to write to CloudWatch Logs



CloudWatch Logs – Log Groups

Log Groups

A collection of log streams. It's common to name logs groups with the forward-slash syntax:

eg.

/exampro/prod/app

/exampro/prod/db

/exampo/dev/app

/exampo/dev/db

Retention setting

Expire events after:

- Never expire
- 1 day
- 3 days
- 5 days
- 1 week (7 days)
- 2 weeks (14 days)
- 1 month (30 days)
- 2 months (60 days)
- 3 months (90 days)
- 4 months (120 days)
- 5 months (150 days)
- 6 months (180 days)
- 12 months (365 days)
- 13 months (400 days)
- 18 months (545 days)
- 24 months (731 days)
- 60 months (1827 days)
- 120 months (3653 days)

Create log group

Log group name

/my-app/prod

Cancel Create

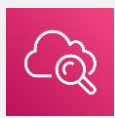
Log groups (102)

Search: /my-app

Exact match 1 match

Log group	Retention
/my-app/prod	Never expire

You can set the retention on a Log Group between Never expire and 120 months (10 years)



CloudWatch Logs – Log Streams

Log Streams

A log stream represents a **sequence of events** from an **application or instance being monitored**.

Create log stream

Log stream name

my-log-stream

Cancel Create

You can create Log Streams manually, but generally, this is automatically done by the service you are using

Here is a Log Group for a **Lambda function**
You can see here the Log Streams are named after the **running instance**. Lambdas frequently run on new instances so the stream streams contain timestamps

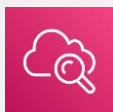
☐	Log stream	Last event time
☐	2020/07/06/[\$LATEST]ebca38579fac4842b531b260d5c35e0e	7/6/2020, 7:41:24 PM
☐	2020/07/06/[\$LATEST]7679ba0f37b14a3da994cd243963ca60	7/6/2020, 6:14:42 PM
☐	2020/07/06/[\$LATEST]bb7edeb95cb345b48dd151a79367a5d6	7/6/2020, 3:52:56 PM
☐	2020/07/06/[\$LATEST]e1544efd95a4492585b9c8d27ddea4b	7/6/2020, 1:30:09 PM
☐	2020/07/06/[\$LATEST]86a8ec4dcff746628effaeb82b41e1cc	7/6/2020, 12:28:00 PM
☐	2020/07/06/[\$LATEST]a06263c73f8242e5a0e35b366b3bbdf9	7/6/2020, 10:08:43 AM

☐	Log stream	Last event time
☐	i-0761fcbdbcb19ffc8	7/6/2020, 6:56:31 PM
☐	i-09239615bc7f3f552	7/5/2020, 9:40:42 PM
☐	i-06c9e4fb3469e17a4	7/5/2020, 9:21:08 PM
☐	i-0450c5ca38bdcd125	7/5/2020, 8:27:30 PM
☐	i-01a34b0a12504edfa	7/5/2020, 12:42:24 AM
☐	i-0e4f5ec7610f21d08	7/5/2020, 12:32:35 AM

Here is a Log Group for an **application log running on EC2**. You can see here the Log Streams are named after the **running instance's Instance ID**

☐	Log stream	Last event time
☐	exampro-events-crawler	6/30/2019, 12:57:11 PM
☐	exampro-waf-logs	6/26/2019, 9:00:49 AM
☐	exampro-leads-crawler	3/24/2019, 7:57:03 PM
☐	dynamodb-events-tracking	3/13/2019, 4:38:00 PM
☐	cloudtrail	2/24/2019, 4:12:18 PM

Here is a Log Group for **AWS Glue**. You can see here the Log Streams are named after the **Glue Jobs**.



CloudWatch Logs – Log Events

Log Events

Represents a single event in a log file. Log events can be seen within a Log Stream.

▶	2020-07-06T20:12:18.079-04:00	START RequestId: e4b5bd10-5d88-4d7b-870c-daf793159b88 Version: \$LATEST
▶	2020-07-06T20:12:18.082-04:00	{"records_size":1}
▶	2020-07-06T20:12:18.093-04:00	{"failed_put_count":0}
▶	2020-07-06T20:12:18.127-04:00	END RequestId: e4b5bd10-5d88-4d7b-870c-daf793159b88
▶	2020-07-06T20:12:18.127-04:00	REPORT RequestId: e4b5bd10-5d88-4d7b-870c-daf793159b88 Duration: 45.32 ms Billed Duration: 100 ms Memory Size: 128

You can use filter events to filter out logs based on simple or Pattern matching syntax:

Log events

🔍 *Filter events*

🔍 D	×	Clear	1m	30m	1h	12h	custom 📅	⚙️
▶	Timestamp	Message						
▶	2020-07-05T21:39:26.857-04:00	D, [2020-07-06T01:39:26.596187 #3979] DEBUG -- : [1m [35m (0.4ms)						
▶	2020-07-05T21:39:26.857-04:00	D, [2020-07-06T01:39:26.614381 #3979] DEBUG -- : [1m [35m (1.5ms)						
▶	2020-07-05T21:39:26.857-04:00	D, [2020-07-06T01:39:26.621670 #3979] DEBUG -- : [1m [36mActiveRec						
▶	2020-07-05T21:39:26.857-04:00	D, [2020-07-06T01:39:26.626819 #3979] DEBUG -- : [1m [35m (0.4ms)						
▶	2020-07-05T21:39:26.857-04:00	D, [2020-07-06T01:39:26.627990 #3979] DEBUG -- : [1m [35m (0.4ms)						



CloudWatch Logs Insights

CloudWatch Logs Insights enables you to **interactively search and analyze your CloudWatch log data** and has the following advantages:

- more robust filtering than using the simple Filter events in a Log Stream
- Less burdensome than exporting logs to S3 and analyzing them via Athena.

CloudWatch Logs Insights supports all types of logs.

CloudWatch Logs Insights is commonly used via the console to do ad-hoc queries against logs groups.

CloudWatch Insights has its own language called:

- CloudWatch Logs Insights **Query Syntax**

```
filter action="REJECT"  
| stats count(*) as numRejections by srcAddr  
| sort numRejections desc  
| limit 20
```

CloudWatch > CloudWatch Logs > Logs Insights [Switch to the original interface.](#)

Select log group(s) 5m 30m 1h 3h 12h custom

```
fields @timestamp, @message  
| sort @timestamp desc  
| limit 20
```

Run query Save History

Logs Visualization Export results Add to dashboard

No results
Run a query to see related events

- A single request can query up to **50 log groups**.
- Queries **time out after 15 minutes**, if they have not been completed.
- Query results are **available for 7 days**.



CloudWatch Logs Insights

AWS provides sample queries that can get you started for common tasks, And to ease learning the Query Syntax. A good example is filtering VPC Flow Logs.

Queries

Help

Saved queries

Create query

Sample queries [Learn more](#)

- ▶ Lambda
- ▼ VPC Flow Logs
 - ▶ Average, min, and max byte transfers by source and destination IP addresses
 - ▶ IP addresses using UDP transfer protocol
 - ▶ Top 10 byte transfers by source and destination IP addresses
 - ▼ **Top 20 source IP addresses with highest number of rejected requests**

```
filter action="REJECT"
| stats count(*) as numRejections by srcAddr
| sort numRejections desc
| limit 20
```


[Apply](#)

5m 30m **1h** 3h 12h custom

Clear

```
filter action="REJECT"
| stats count(*) as numRejections by srcAddr
| sort numRejections desc
| limit 20
```

Run query Save History

Logs Visualization

Export results Add to dashboard

Showing 20 of 1,709 records matched ⓘ [Hide histogram](#)

15,067 records (2.0 MB) scanned in 4.6s @ 3,281 records/s (452.0 kB/s)

#	srcAddr	numRejections
▶ 1	185.154.13....	52
▶ 2	45.227.255....	51
▶ 3	87.251.74.62	49
▶ 4	94.102.51.28	33
▶ 5	52.41.160.1...	30

You can create and save your own queries to make future repetitive tasks easier.



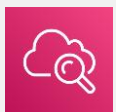
CloudWatch Insights – Discovered Fields

When CloudWatch Insights reads logs, it will first analyze the log events and try to structure the content by generating fields that you can then use in your query.

CloudWatch Logs Insights inserts the **@** symbol at the start of fields that it generates.

Five system fields will be automatically generated:

- 
- @message** — the raw unparsed log event.
 - @timestamp** — the event timestamp contained in the log event's timestamp field.
 - @ingestionTime** — the time when the log event was received by CloudWatch Logs.
 - @logStream** — the name of the log stream that the log event was added to.
 - @log** — is a log group identifier in the form of account-id:log-group-name.



CloudWatch Insights – Discovered Fields

CloudWatch Logs Insights **automatically discovers** fields in logs from AWS services such as:



Amazon VPC Flow Logs

@timestamp, @logStream, @message, accountId, endTime, interfaceId, logStatus, startTime, version, action, bytes, dstAddr, dstPort, packets, protocol, srcAddr, srcPort



Amazon Route 53

@timestamp, @logStream, @message, edgeLocation, hostZoneId, protocol, queryName, queryTimestamp, queryType, resolverIp, responseCode, version



AWS Lambda

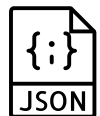
@timestamp, @logStream, @message, @requestId, @duration, @billedDuration, @type, @maxMemoryUsed, @memorySize With X-Ray: @xrayTraceId and @xraySegmentId



AWS CloudTrail

eventVersion, eventTime, eventSource, eventName, awsRegion, sourceIPAddress, userAgent ...

There are more, you have to check the JSON in the CloudTrail events to see full list



JSON Logs

The fields of a JSON log will be turned into fields



Other Types of Logs

Fields that CloudWatch Logs Insights doesn't automatically discover

you can use the **parse** command to extract and create ephemeral fields for use in that query.



CloudWatch Insights – Discovered Fields

You can see all the **discovered fields** by CloudWatch Insights here:

The screenshot shows the CloudWatch Logs Insights interface. The top navigation bar includes 'CloudWatch', 'CloudWatch Logs', and 'Logs Insights'. A link to 'Switch to the original interface.' is also present. The main panel is divided into two sections: 'Logs' and 'Visualization'. The 'Logs' section shows a query for 'exampro-flow-logs' with a filter action of 'REJECT'. The 'Visualization' section shows a histogram of 'numRejections' over time, with a table of results below it. The 'Discovered fields' panel is open on the right, listing various fields and their discovery percentages. A red arrow points from the text 'discovered fields' to this panel.

Discovered fields

Learn more [\[link\]](#)

Search for a field

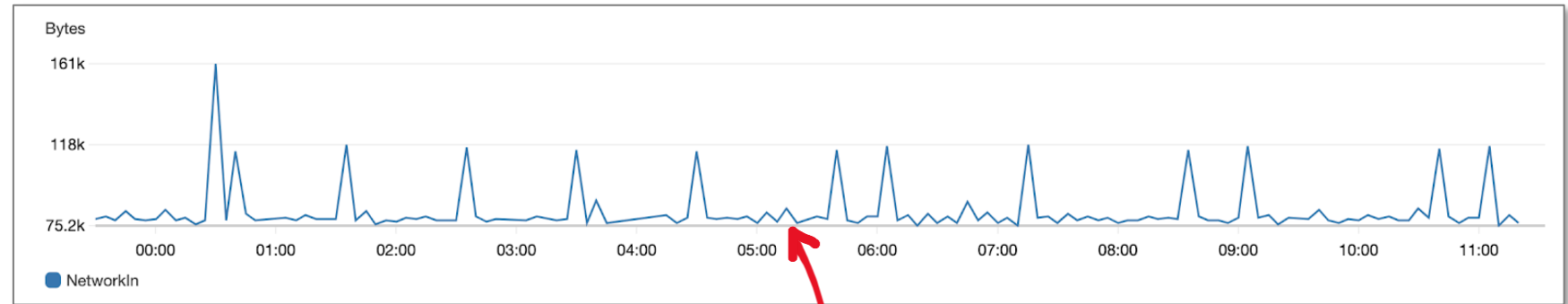
▶ @ingestionTime	100%
▶ @logStream	100%
▶ @message	100%
▶ @timestamp	100%
▶ accountId	100%
▶ action	100%
▶ bytes	100%
▶ dstAddr	100%
▶ dstPort	100%
▶ end	100%
▶ interfaceId	100%
▶ logStatus	100%
▶ packets	100%
▶ protocol	100%
▶ srcAddr	100%
▶ srcPort	100%
▶ start	100%
▶ version	100%



CloudWatch Metrics

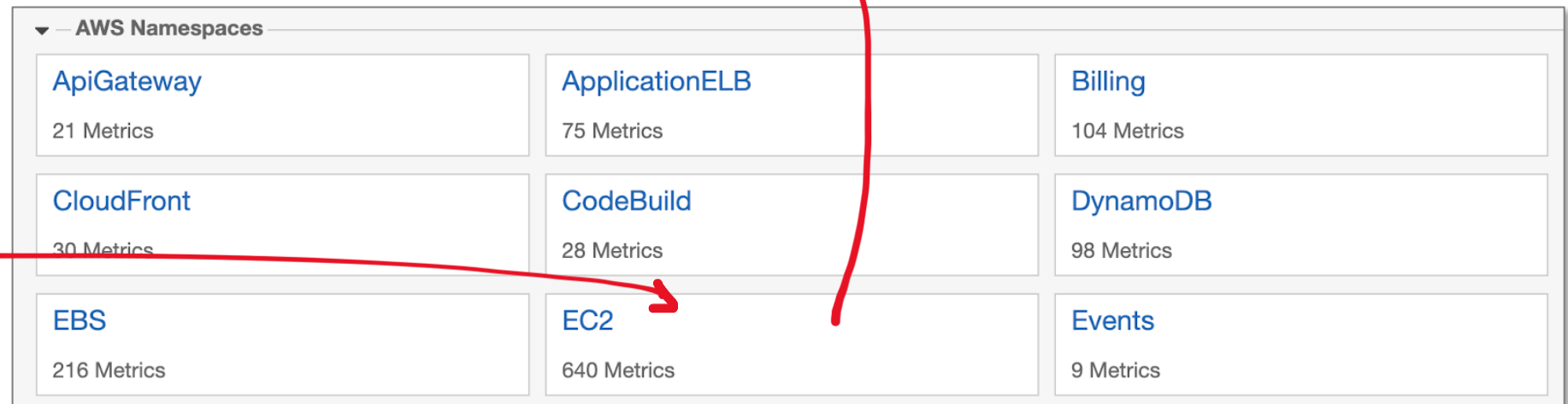
A CloudWatch Metric represents a **time-ordered set of data points**
It's a **variable** that is **monitored over time**.

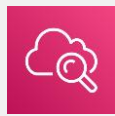
CloudWatch comes with many **predefined** metrics that are generally name-spaced by AWS Service.



EC2 Per-Instance Metrics

- CPUUtilization
- DiskReadOps
- DiskWriteOps
- DiskReadBytes
- DiskWriteBytes
- **NetworkIn**
- NetworkOut
- NetworkPacketsIn
- NetworkPacketsOut





CloudWatch Metrics

- Metrics are data about the performance of your systems.
- By default, several services provide free metrics for resources (EC2, EBS, RDS)
- Enable detailed monitoring for some resources
- publish your own application metrics
- Metric data is kept for 15 months, enabling you to view both up-to-the-minute data and historical data.



CloudWatch – Availability of Data

When an AWS Service emits data to CloudWatch, the availability of the data varies based on the AWS Service:



Elastic Compute Cloud (EC2) Other Services

Basic Monitoring	5 minute interval	1 minute / 3 minute / 5 minute
Detailed Monitoring	1 minute interval	N/A

Majority of AWS services data availability is 1 minute



CloudWatch Agent and Host Level Metrics

Certain metrics that may seem to be tracked by default for EC2 instances actually require installing the **CloudWatch Agent** to monitor

Host Level Metrics

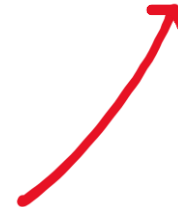
These are what you get **without installing** the Agent

- CPU Usage
- Network Usage
- Disk Usage
- Status Checks
 - Underlying Hypervisor status
 - Underlying EC2 instance status

Agent Level Metrics

These are what you get **when installing** the Agent

- **Memory utilization**
- Disk Swap utilization
- **Disk Space utilization**
- Page file utilization
- Log collection



The CloudWatch Agent **is also used to collect various logs** from an EC2 instance **and send them to a CloudWatch Log Group**



Installing the CloudWatch Agent



The CloudWatch Agent can be installed using **AWS Systems Manager (SSM) Run Command** onto the target EC2 instance.

Run a Command

AWS-ConfigureAWSPackage Amazon Windows, Linux

Command parameters

Action

(Required) Specify whether or not to install or uninstall the package.

Install

Installation Type

Uninstall and reinstall

Name

(Required) The package to install/uninstall.

AmazonCloudWatchAgent

Version

latest

AWS-ConfigureAWSPackage

Description

Content

Versions

Details

Description

Install or uninstall a Distributor package. You can install the latest version, default version, or a version of the package you specify. Packages provided by AWS such as AmazonCloudWatchAgent, AwsEnaNetworkDriver, and AWSPVDriver are also supported.

Choose which EC2 instances you want to install the Agent on.

Targets

Choose a method for selecting targets.

☐

Specify instance tags

Specify one or more tag key-value pairs to select instances that share those tags.

☒

Choose instances manually

Manually select the instances you want to register as targets.

☐

Choose a resource group

Choose a resource group that includes the resources you want to target.

You must attach the **CloudWatchAgentServerRole** IAM role to the EC2 instance to be able to run the agent on the instance.



Custom Metrics and High-Resolutions

You can publish your own **Custom Metrics** using the AWS CLI or SDK.

```
aws cloudwatch put-metric-data \  
  --metric-name Enterprise-D \  
  --namespace Starfleet \  
  --unit Bytes \  
  --value 231213412 \  
  --dimensions HullIntegrity=100, Shield=70, Thrusters=maximum
```

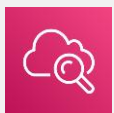
High-Resolution Metrics

When you publish a custom metric, you can define the resolution as either:

- standard resolution (1 minute)
- **high resolution (> 1 minute to 1 second)**

With High-Resolution, you can track in intervals of:

- **1 second**
- 5 seconds
- 10 seconds
- 30 seconds
- multiple of 60 seconds.



CloudWatch Agent – Log Collection

The CloudWatch Agent **can send logs running on your EC2 instance** to a CloudWatch Log Group.

To send logs:

1. the Agent Configuration needs to be updated to include the logs
2. the CloudWatch Agent service needs to be restarted.

The Agent's configuration file is located at `/etc/awslogs/awslogs.conf`

```
[exampro_application_log]
log_group_name = /exampro/rails/logs/production
log_stream_name = {instance_id}
datetime_format = %Y-%m-%dT%H:%M:%S.%f
file = /var/www/my-app/current/log/production.log*
```

You specify the location of the log file and what log group you want the log to be sent to

```
sudo service awslogsd stop
sudo service awslogsd start
```

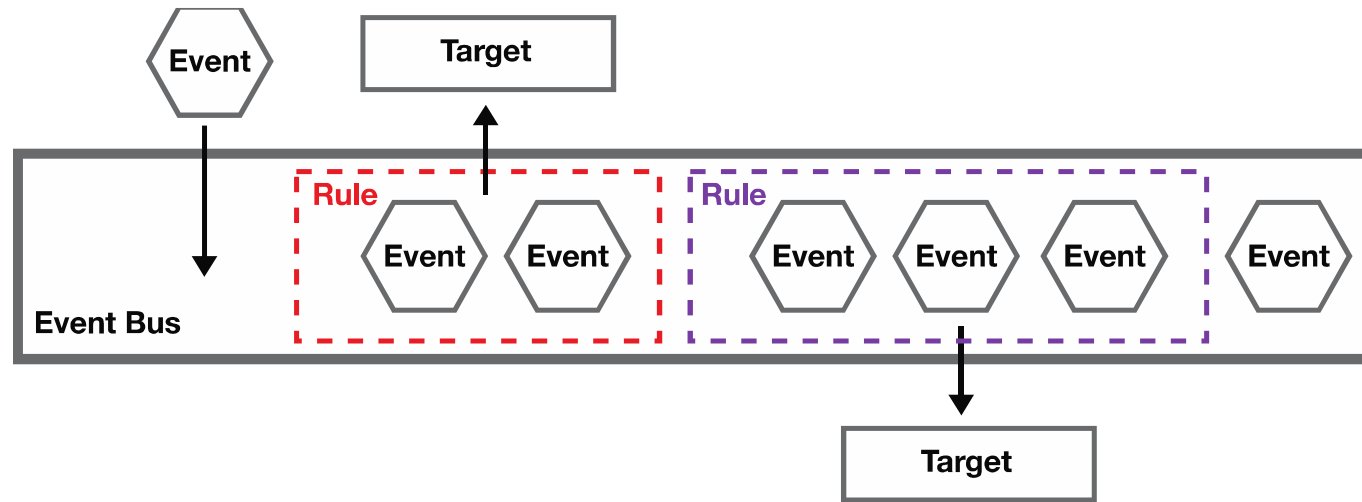
Restart CloudWatch Agent so it will send the added log files



Introduction to Amazon EventBridge

What is an Event Bus?

An event bus **receives events** from a **source** and **routes events** to a **target** based on **rules**



EventBridge is a **serverless** event bus service that is used for application integration by **streaming real-time** data to your applications

EventBridge was *formerly* called **Amazon CloudWatch** Events.



EventBridge Core Components

Event Bus

Holds event data and defines rules on an event bus to react to events.

Default Event Bus — An AWS account has a default event bus

Custom Event Bus — Scoped to multiple accounts or other AWS accounts

SaaS Event Bus — Scoped to third-party SaaS Providers

Rules

Determines what events to capture and pass to targets. (100 Rules per bus)

Targets

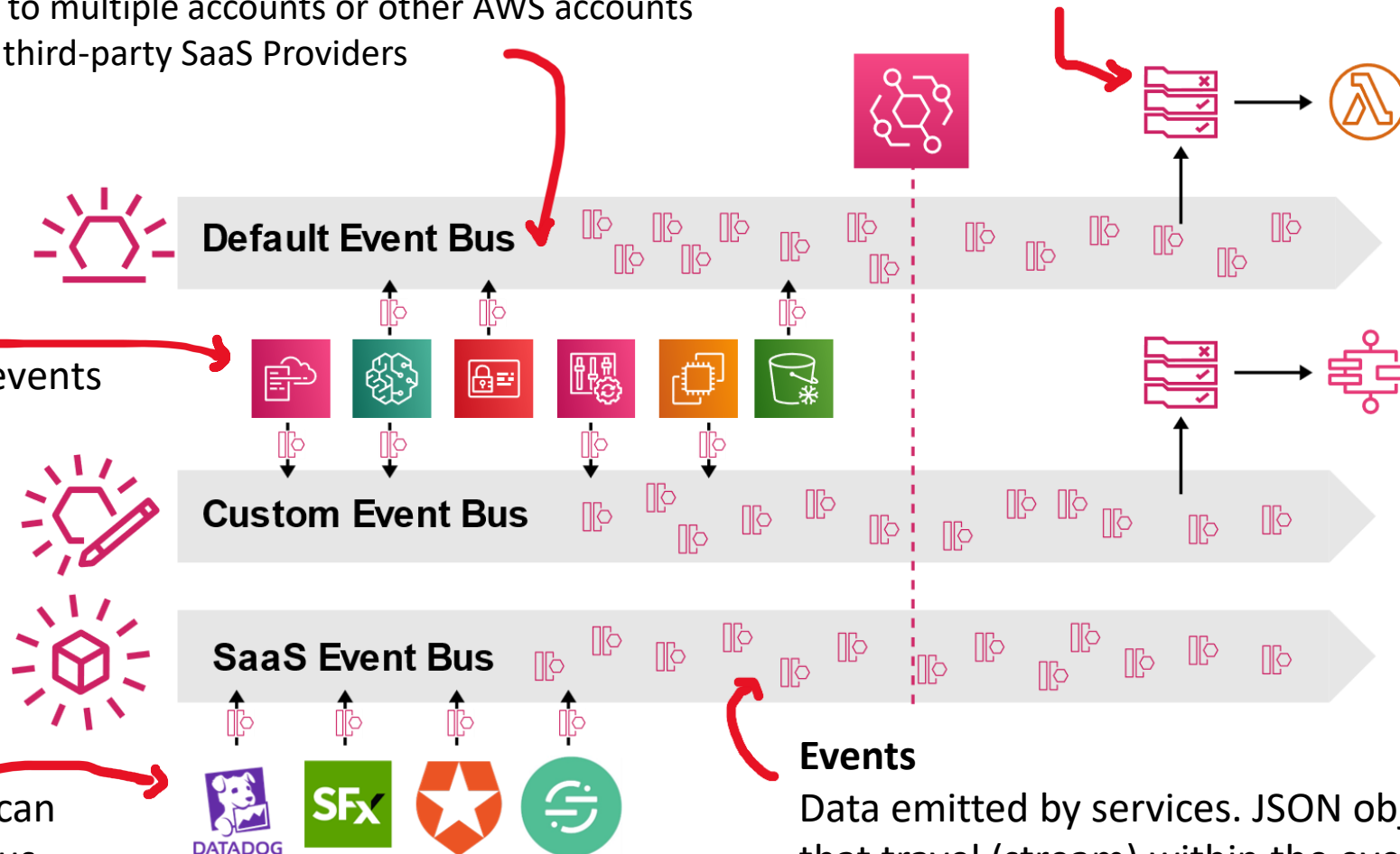
AWS Services that consume events
(5 targets per rule)

Producers

AWS Services that emit events

Partner Sources

Are third-party apps that can emit events to an event bus





Amazon EventBridge – The Anatomy of an Event

The top-level fields listed here will always appear in every single event.

The contents of fields appearing under **detail** will vary based on what AWS Cloud service emits the event.

version By default, this is set to 0 (zero) in all events.

id A unique value is generated for every event.

detail-type Identifies fields and values that appear in the detail field

source Identifies the service that sourced the event.

account 12-digit number identifies the AWS account.

time the event timestamp

region AWS region where the event originated

resources JSON array contains ARNs that identify resources that are involved in the event

detail JSON object containing data provided by the Cloud Service. Can contain 50 fields nested several levels deep

```
{
  "version": "0",
  "id": "6a7e8feb-b391-4ef7-e9f1-bf3703467718",
  "detail-type": "EC2 Instance State-change Notification",
  "source": "aws.ec2",
  "account": "121212121212",
  "time": "2020-05-22T14:22:48Z",
  "region": "us-east-1",
  "resources": [
    "arn:aws:ec2:us-east-1:123456789012:instance/i-1234567890abcdef0"
  ],
  "detail": {
    "instance-id": "i-1234567890abcdef0",
    "state": "terminated"
  }
}
```



Amazon EventBridge – Scheduled Expressions

You can create EventBridge Rules that trigger on a schedule. You can think of it as **Serverless Cron Jobs**

- All scheduled events use **UTC time zone**
- the minimum precision for schedules is **1 minute**.

EventBridge supports **cron expressions** and **rate expressions**

Cron expression

Very fine grain control

☒ Cron expression

0 10 * * ? *

Mins	Hours	Day of Month	Day of Week	Year	Run this event:
15	12	*	MON-FRI	*	12:15 pm (UTC) Monday to Friday
0/15	*	*	?	*	every 15 minutes

Rate expressions

Easy to set, not as fine-grained

☒ Fixed rate every

8 Hours

Hours

Minutes

Hours

Days



Amazon EventBridge - Rules

You specify **up to five Targets** for a single rule. Commonly targeted AWS Cloud Services:

- Lambda Function
- SQS queue
- SNS topic
- Firehose delivery stream
- ESC Task

You may have some additional fields to **select the target**.

Eg.

- Lambda Function
- Lambda Alias
- Lambda Version

Target Remove

Lambda function ▼

Function

starTrekKaraokeVideoProcessing ▼

Target

Select target(s) to invoke when an event matches your rule)

Lambda function

API Gateway

Batch job queue

CloudWatch log group

CodeBuild project

CodePipeline

EC2 CreateSnapshot API call

EC2 RebootInstances API call

EC2 StopInstances API call

EC2 TerminateInstances API call

ECS task

Event bus in another AWS account

Firehose delivery stream

Inspector assessment template

Kinesis stream

Lambda function

SNS topic

SQS queue

SSM Automation

SSM OpsItem

SSM Run Command

Step Functions state machine

You can specify what gets passed along by changing **Configure Input**. This acts as a filter.

▼ Configure input

☒ Matched events [Info](#)

☐ Part of the matched event [Info](#)

☐ Constant (JSON text) [Info](#)

☐ Input transformer [Info](#)

Match Events

The entire event pattern text is passed to the target when the rule is triggered. (Just Pass everything)



Amazon EventBridge – Configure Input

Match Events The entire event pattern text is passed to the target when the rule is triggered.

Part of the matched event
only the part of the event text that you specify is passed to the target.

☒ Part of the matched event [Info](#)

Constant (JSON text)

send static content instead of the matched event data. (Mocked JSON)

☒ Constant (JSON text) [Info](#)

Input Transformer

You can transform the event text into a different format of a **string** or a **JSON object**

You can map fields from the event data to variables. Then you can use those variables in a string or JSON object, and that is what gets passed along.

You **can't use these** as variable names (reserved by AWS):

- aws.events.rule-arn
- aws.events.rule-name
- aws.events.event

☒ Input transformer [Info](#)

\"state\" : \"\$.detail.state\"
}"/>

☒ Input transformer [Info](#)

\"state\" : \"\$.detail.state\"
}"/>

\"state\": <state>
}"/>



Amazon EventBridge – Schema Registry

EventBridge Schema Registry allows you to **create**, **discover** and **manage OpenAPI schemas** for events on EventBridge.

What is a schema?

A schema is an outline, diagram, or model. Schemas are often used to describe the structure of different types of data.

Catalog changes to the schema

```
{
  "version": "0",
  "id": "6a7e8feb-b391-4ef7-e9f1-bf3703467718",
  "detail-type": "EC2 Instance State-change Notification",
  "source": "aws.ec2",
  "account": "121212121212",
  "time": "2020-05-22T14:22:48Z",
  "region": "us-east-1",
  "resources": [
    "arn:aws:ec2:us-east-1:123456789012:instance/i-1234567890abcdef0"
  ],
  "detail": {
    "instance-id": "i-1234567890abcdef0",
    "state": "terminated"
  }
}
```

Search all schemas

Q instance

[aws.ec2@EC2InstanceStateChange...](#)

AWS event schema registry

Found in [version 1](#)

View the schema and Download the Code Bindings

Version 1 Created on Nov 30, 2019, 07:11 PM EST

Action ▼

Download code bindings

```
18     },
19     "detail-type": {
20       "type": "string"
21     },
22     "resources": {
23       "type": "array",
24       "items": {
25         "type": "string"
26       }
27     },
28     "id": {
29       "type": "string"
30     },
31     "source": {
32       "type": "string"
```

It will monitor events within the Event Bus



Amazon EventBridge – Schema Registry

Why would you want **a schema** of the events in your EventBridge event bus?

So you can see if the structure of the events have **changed over time**.

[aws.autoscaling@AWSAPICallViaCloudTrail](#)

AWS event schema registry **1 version**

Last updated Nov 30, 2019, 07:11 PM EST

This makes it easier for developers to know
What data to expect from a type of event
so it's easier to integrate into applications.

So we can download Code Bindings for various languages to make it easier for developers to work with events in their code.

```
1  import { EC2InstanceLaunchLifecycleAction } from './schema/  
    aws/autoscaling/ec2instancelaunchlifecycleaction/  
    EC2InstanceLaunchLifecycleAction'  
2  
3  const event = new EC2InstanceLaunchLifecycleAction()  
4  event.ec2InstanceId
```

 **ec2InstanceId** (property) EC2Insta

A **Code Binding** is when the schema is wrapped in a programming Object. This standardizes how to work with event data in the code Leading to fewer bugs and easier discovery of data.

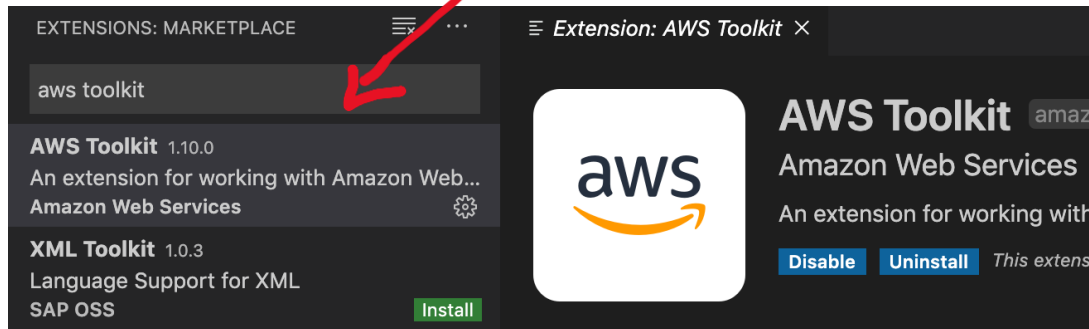


Amazon EventBridge – Schema Registry

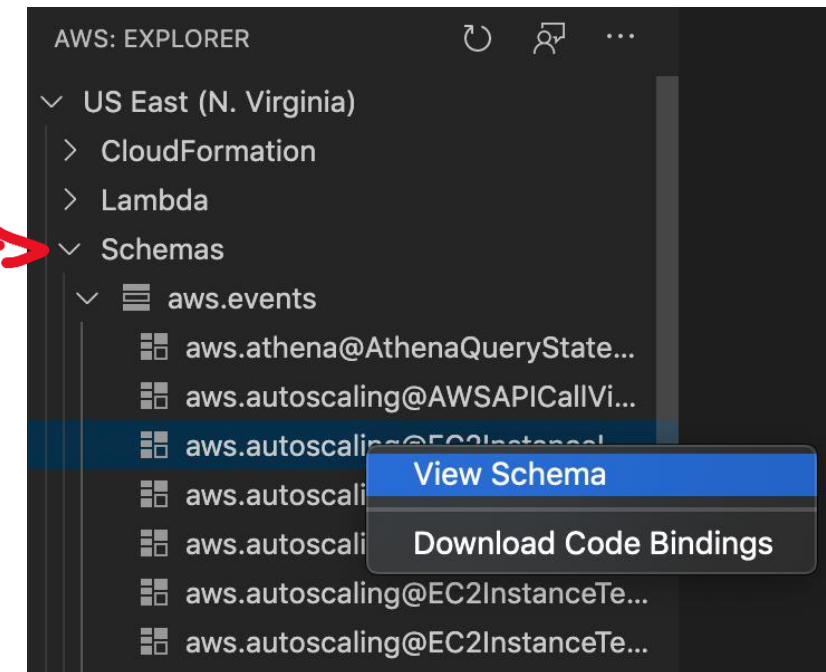
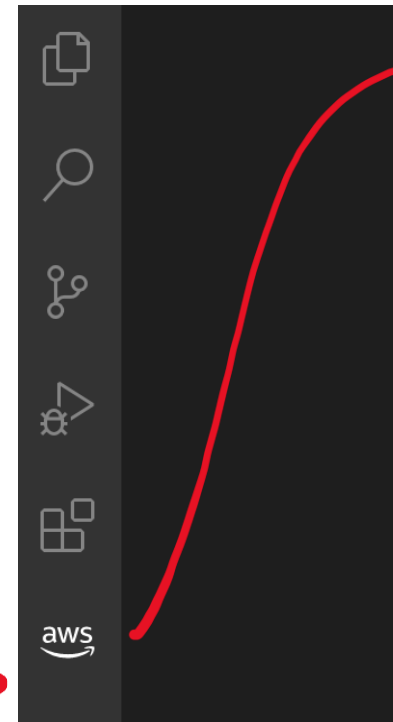
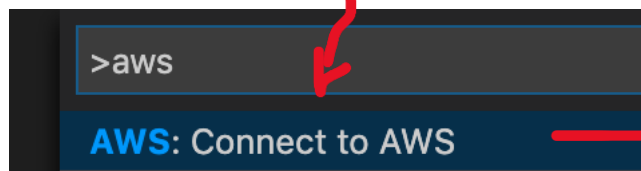
By installing **the AWS Toolkit** for VSCode, you can easily **View Schemas** and install **Code Bindings**



VSCode is a very popular code editor.
You can install **“extensions”** directly with the editor.
This is how you would install AWS Toolkit



Once installed, you open the Command Pallet (Command +Shift + P) and **Connect to your AWS Account**

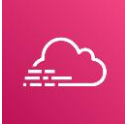


Then you will have an AWS **sidebar**



Amazon EventBridge – CloudTrail Events

Not all AWS Services emit CloudWatch Events.

For other AWS Services, we can use  **CloudTrail**

Turning on CloudTrail allows EventBridge to track changes to AWS Services made by API calls or by AWS users.

The Detail Type of CloudTrail will be called:
“AWS API Call via CloudTrail”

AWS API call events that are **larger than 256 KB** in size are not supported.



```
{
  ...
  "detail-type": "AWS API Call via CloudTrail",
  "detail": {
    "eventVersion": "1.03",
    "userIdentity": {
      "type": "Root",
      "principalId": "123456789012",
      "arn": "arn:aws:iam::123456789012:root",
      "accountId": "123456789012",
      "sessionContext": {
        "attributes": {
          "mfaAuthenticated": "false",
          "creationDate": "2020-02-20T01:05:59Z"
        }
      }
    }
  },
  "eventTime": "2020-02-20T01:09:13Z",
  "eventSource": "s3.amazonaws.com",
  "eventName": "CreateBucket",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "100.100.100.100",
  "userAgent": "[S3Console/0.4]",
  "requestParameters": {
    "bucketName": "my-bucket"
  },
  "responseElements": null,
  "requestID": "9D767BCC3B4E7487",
  "eventID": "24ba271e-d595-4e66-a7fd-9c16cbf8abae",
  "eventType": "AwsApiCall"
}
```



Amazon EventBridge – Event Patterns

Event Patterns are used to **filter what events should be used** to pass along to a target.

You can filter events by providing **the same fields and values** found in the original Events.

Let us say we want to create a rule that Only reacts when an EC2 instance is terminated.

```
{
  "version": "0",
  "id": "6a7e8feb-b491-4cf7-a9f1-bf3703467718",
  "detail-type": "EC2 Instance State-change Notification",
  "source": "aws.ec2",
  "account": "111122223333",
  "time": "2020-12-22T18:43:48Z",
  "region": "us-east-1",
  "resources": [
    "arn:aws:ec2:us-east-1:123456789012:instance/i-1234567890abcdef0"
  ],
  "detail": {
    "instance-id": "i-1234567890abcdef0",
    "state": "terminated"
  }
}
```

We would just supply the following as the **Event Pattern**

```
{
  "source": [ "aws.ec2" ],
  "detail-type": [ "EC2 Instance State-change Notification" ],
  "detail": {
    "state": [ "terminated" ]
  }
}
```

Event pattern [Info](#)
Build a pattern to match events

Event matching pattern
You can use pre-defined pattern provided by a service or create a custom pattern

☐ Pre-defined pattern by service
☒ Custom pattern

Event pattern Save Cancel

```
{
  "source": [
    "aws.ec2"
  ],
  "detail-type": [
    "EC2 Instance State-change Notification"
  ],
  "detail": {
    "state": [
      "terminated"
    ]
  }
}
```



Amazon EventBridge – Event Patterns

Prefix Matching

match on the prefix of a value in the event source

```
"region": [ { "prefix": "ca-" } ]
```

Anything-but Matching

matches anything except what's provided in the rule

```
"state": [ { "anything-but": [ "stopped", "overloaded" ] } ]
```

Numeric Matching

Matches against numeric operator for "<", ">", "=", "<=", ">="

```
"x-limit": [ { "numeric": [ ">", 0, "<=", 5 ] } ]
```

IP Address Matching

matching against available for both IPv4 and IPv6 addresses

```
"source-ip": [ { "cidr": "10.0.0.0/24" } ]
```

Exists Matching

matching works on the presence or absence of a field in the JSON

```
"c-count": [ { "exists": false } ]
```

Empty Value Matching

For strings, you can use "" to match empty

For other values, you can null

```
"eventVersion": [ "" ]
```

```
"responseElements": [ null ]
```

Complex Example with Multiple Matching

combine multiple matching rules into a more complex event pattern

```
{
  "time": [ { "prefix": "2017-10-02" } ],
  "detail": {
    "state": [ { "anything-but": "initializing" } ],
    "c-count": [ { "numeric": [ "<", 10 ] } ],
    "x-limit": [ { "anything-but": [ 100, 200, 300 ] } ]
  }
}
```



Amazon EventBridge – Partner event sources

A list of third-party service providers can be integrated to work with EventBridge.
Events will emit from the service provider into your Event Bus

Service provider
AWS services or custom/partner services

Service partners ▼

Service name
The name of partner service selected as the event source

Select a service ▲

Q |

- Atlassian - Opsgenie
- Auth0
- Blitline
- BUIDLhub
- Buildkite
- CleverTap
- Datadog
- Epsagon
- Freshworks
- Game Server Services Co., Ltd.
- Kloudless
- Mackerel
- MongoDB

Zendesk's Events Connector for AWS enables streaming of event data from Zendesk's support suite into Amazon EventBridge... Learn more Set up	PagerDuty's platform for real-time IT operations integrates with Amazon EventBridge to enable teams to take action on... Learn more Set up	Datadog's monitoring platform allows users to route triggered alerts to notification channels with rich contextual informatio... Learn more Set up	The Whispir platform excels at delivering streamlined communication processes for businesses. Our integration wit... Learn more Set up
Saviynt enables enterprises to secure applications, data and infrastructure in a single platform for cloud and... Learn more Set up	The Segment Destination for Amazon EventBridge is a new way of getting your first-party customer data into AWS. While... Learn more Set up	SignalFx, the only real-time cloud monitoring platform for infrastructure, microservices, and applications, collects and... Learn more Set up	SugarCRM leverages Amazon EventBridge to establish a secure and highly configurable stream of notifications between its... Learn more Set up



CloudWatch Alarms

A CloudWatch Alarm monitors a **CloudWatch Metric** based on a **defined threshold**.

☐	Name	State	Last state update	Conditions	Actions
☐	Network_In	In alarm	2020-07-20 13:01:35	NetworkIn > 300 for 1 datapoints within 5 minutes	No actions

When the alarm is breached (goes outside the defined threshold), then it changes **state**.

When it changes state, we can define what **action it should trigger**.

Metric Alarm States

- **OK** The metric or expression is **within** the defined threshold
- **ALARM** The metric or expression is **outside** of the defined threshold
- **INSUFFICIENT_DATA**
 - The alarm has **just started**
 - the metric is **not available**
 - **Not enough data** is available

Notification

Alarm state trigger
Define the alarm state that will trigger this action.

☒ **In alarm**
The metric or expression is outside of the defined threshold.

☐ **OK**
The metric or expression is within the defined threshold.

☐ **Insufficient data**
The alarm has just started or not enough data is available.

Select an SNS topic
Define the SNS (Simple Notification Service) topic that will receive the notification.

☒ Select an existing SNS topic

☐ Create new topic

☐ Use topic ARN

Add notification

Auto Scaling action

Add Auto Scaling action

EC2 action

This action is only available for EC2 Per-Instance Metrics.

Add EC2 action

- Notification
- Auto Scaling Group
- EC2 Action



CloudWatch Alarm – Anatomy of an Alarm

Threshold Condition

Defines when a data point is breached

Threshold type

☒ Static
Use a value as a threshold

Whenever NetworkIn is...
Define the alarm condition.

☐ Greater
> threshold

☐ Greater or Equal
≥ threshold

than...
Define the threshold value.

100000

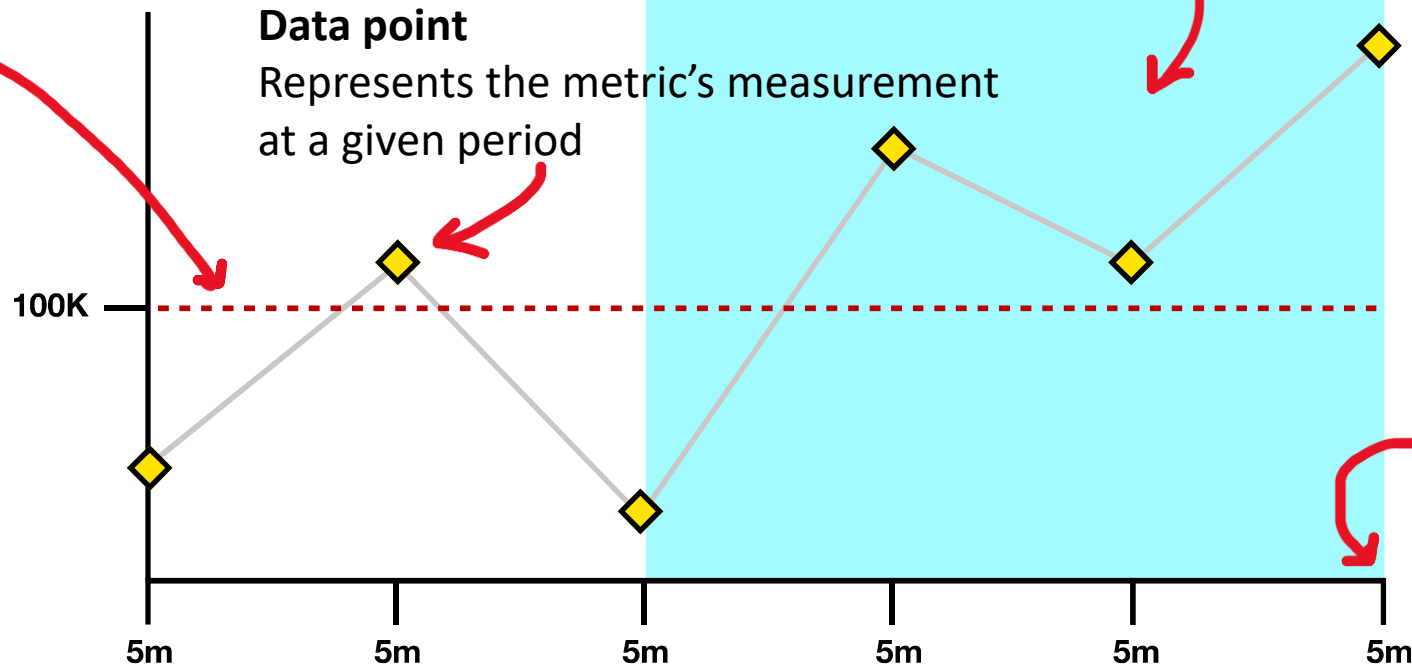
Must be a number

Metric

The actual data
we are measuring

NetworkIn

The volume of incoming network traffic.
Measured in Bytes. When using 5min monitoring,
divide by 300 to get Bytes/second



Datapoints to alarm

1 data point is breached in an evaluation
period going back 4 periods.
This is what triggers the alarm

Datapoints to alarm

Define the number of datapoints within the evaluation period that must be
breaching to cause the alarm to go to ALARM state.

1 out of 4



CloudWatch Alarms – Conditions

When you create an alarm, you define the **threshold**.
The most common type is a **Static Threshold**.

Then you define the condition of the alarm

Then you define the threshold value

Conditions

Threshold type

☒ **Static**
Use a value as a threshold

☐ **Anomaly detection**
Use a band as a threshold

Whenever NetworkIn is...
Define the alarm condition.

☐ **Greater**
> threshold

☐ **Greater/Equal**
≥ threshold

☐ **Lower/Equal**
≤ threshold

☒ **Lower**
< threshold

than...
Define the threshold value.

100000

Must be a number

Example:

You create a CloudWatch Alarm because you want to avoid unexpected charges (Billing Alarm).

- You use the **EstimatedCharges** metric
- You set the Threshold Type to **Static**
- Set the Alarm condition to **Greater**
- Set the threshold value of 50 USD

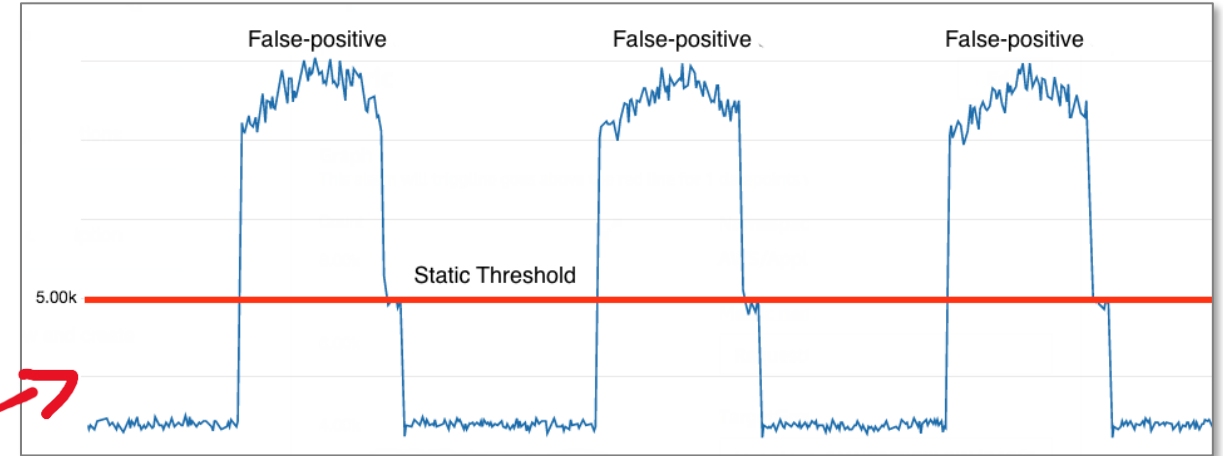


CloudWatch Alarms - Conditions

You may have reoccurring data points that breach a static threshold, but this would not be considered “unusual behavior”

eg. Every morning, all your company employees log in so you have a spike of traffic for 30 mins

Using a Static Threshold Type would trigger the ALARM State, and these would be **false positives**.



Conditions

Threshold type

☐ Static
Use a value as a threshold

☒ Anomaly detection
Use a band as a threshold

Whenever NetworkIn is...

Define the alarm condition

☒ Outside of the band
> or < threshold

☐ Greater than the band
> threshold

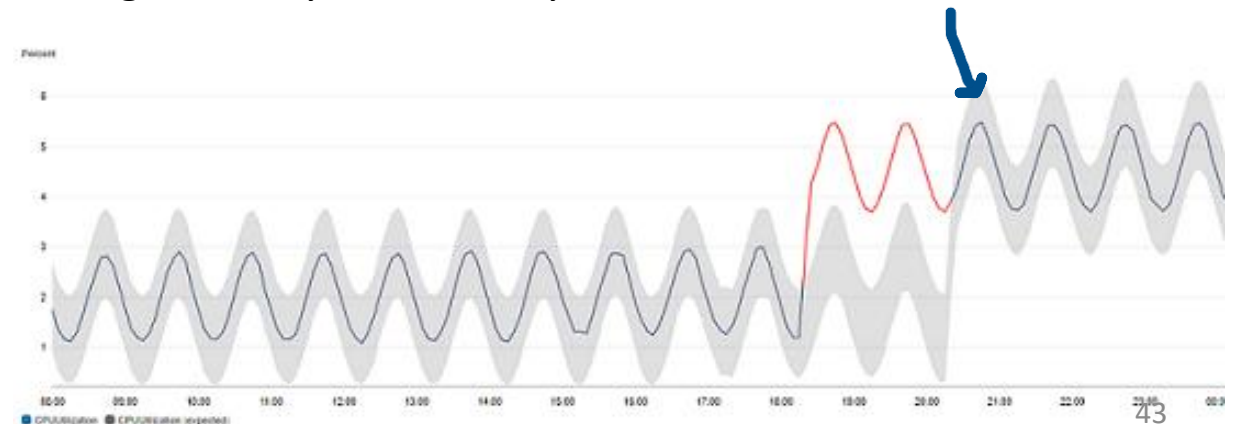
☐ Lower than the band
< threshold

Anomaly detection threshold

Based on a standard deviation. Higher number means thicker band, lower number means thinner band.

Must be a positive number

Using Anomaly detection, you can define a **band** as the threshold





CloudWatch Alarms - Conditions

Period

- 6 hours ▲
- 10 seconds
- 30 seconds
- 1 minute
- 5 minutes
- 15 minutes
- 1 hour
- 6 hours
- 1 day
- Custom

☐ Lower/Equal ☐ Lower

Conditions

Threshold type

☒ Static

Use a value as a threshold

☐ Anomaly detection

Use a band as a threshold

Whenever EstimatedCharges is...

Define the alarm condition.

☒ Greater

> threshold

☐ Greater/Equal

>= threshold

☐ Lower/Equal

<= threshold

☐ Lower

< threshold

than...

Define the threshold value.

50

USD

Must be a number

Datapoints to alarm

Define the number of datapoints within the evaluation period that must be breaching to cause the alarm to go to ALARM state.

1

out of

4

Missing data treatment

How to treat missing data when evaluating the alarm.

Treat missing data as missing ▲

Treat missing data as missing

Treat missing data as good (not breaching threshold)

Treat missing data as ignore (maintain the alarm state)

Treat missing data as bad (breaching threshold)



CloudWatch Alarms – Composite Alarms

Composite Alarms are alarms **that watch other alarms.**

Using composite alarms can help you **reduce alarm noise.**

Imagine you have two alarms, and you configure them to have **no action**:

- CPU Utilization
- NetworkIn

	Name	State	Last state update	Conditions	Actions
☒	Network_In	⚠ In alarm	2020-07-20 13:01:35	NetworkIn > 300 for 1 datapoints within 5 minutes	No actions
☒	CPU_Utilization	✅ OK	2020-07-20 13:00:29	CPUUtilization > 1 for 1 datapoints within 5 minutes	No actions

You **select both Alarms** and create a **Composite Alarm**

Create composite alarm

Then you set the alarm conditions

Conditions

Composite alarm conditions

This alarm will go in alarm when the following rule is met. Configure by using AND/OR in the text editor. [Info](#)

Add another alarm

```
1 ALARM("CPU_Utilization") OR
2 ALARM("Network_In")
```

CancelNext

Alarm state trigger

Define the alarm state that will trigger this action.

☒ In alarm
The alarm rule evaluates to true

☐ OK
The alarm rule evaluates to false

Select an SNS topic

Define the SNS (Simple Notification Service) topic that will receive the notification.

☒ Select an existing SNS topic

☐ Create new topic

☐ Use topic ARN

Configure actions

Notification

Add notification

Auto Scaling action

Add Auto Scaling action

EC2 action

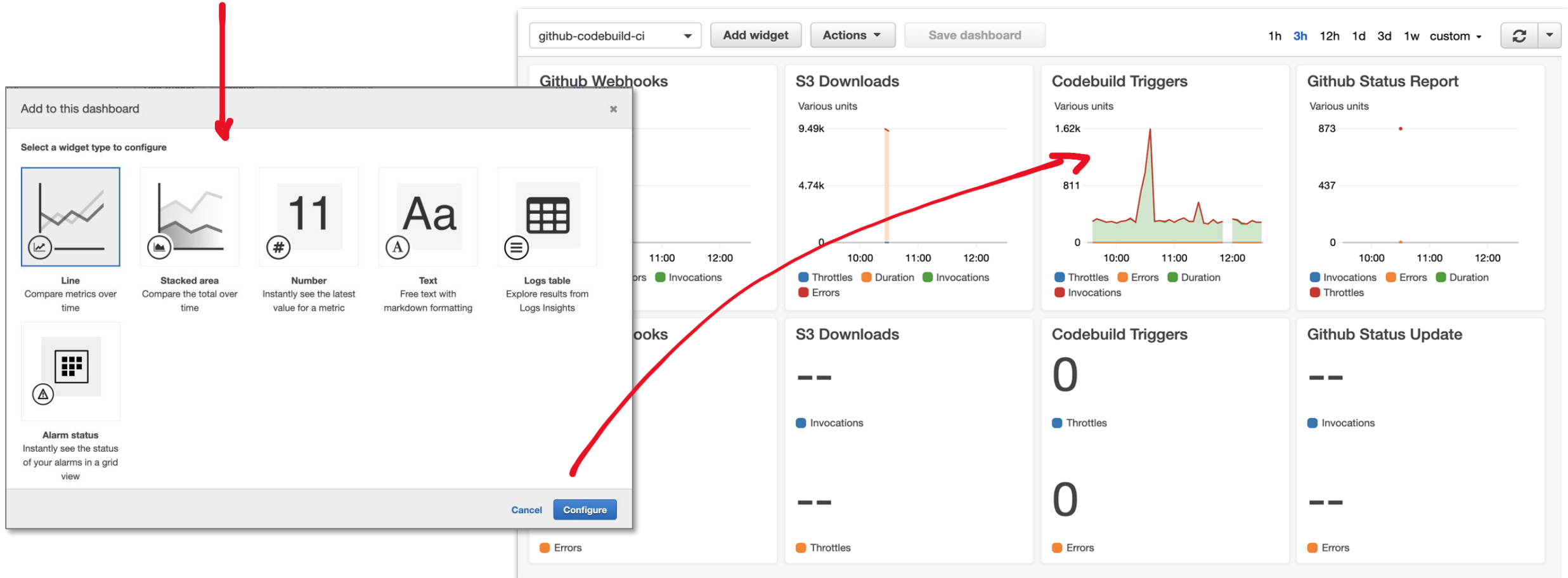
Add EC2 action

The only action you can configure for a composite alarm is an **SNS Topic**

CloudWatch Dashboards

CloudWatch Dashboards allow you to visualize your Cloud Metrics in the form of various graphs.

You create a **widget**, choose and configure a metric and add it to your dashboard



The screenshot displays the AWS CloudWatch console interface. At the top, a dashboard titled 'github-codebuild-ci' is visible, featuring several widgets: 'Github Webhooks', 'S3 Downloads', 'Codebuild Triggers', 'Github Status Report', and 'Github Status Update'. A modal window titled 'Add to this dashboard' is open in the foreground, showing options to select a widget type to configure. The modal includes a red arrow pointing to the 'Add widget' button in the dashboard header and another red arrow pointing to the 'Codebuild Triggers' widget in the dashboard. The modal lists the following widget types:

- Line**: Compare metrics over time
- Stacked area**: Compare the total over time
- Number**: Instantly see the latest value for a metric
- Text**: Free text with markdown formatting
- Logs table**: Explore results from Logs Insights
- Alarm status**: Instantly see the status of your alarms in a grid view

The modal also includes 'Cancel' and 'Configure' buttons at the bottom.



CloudWatch ServiceLens

CloudWatch ServiceLens gives you **observability** for your **distributed applications** by consolidating **metrics, traces, logs, and alarms** into one unified dashboard.

What is a distributed application?

Also known as a distributed system, is when network isolated services or applications that have to communicate over a network, together make a larger system/application.

Applications that could be defined as distributed systems generally utilize:

- Microservices
- Containers
- Various Cloud Services, Compute and Databases tied together using Application Integration Services



ServiceLens **integrates CloudWatch with X-Ray** to provide an **end-to-end view** of your application to help you efficiently:

- **pinpoint performance bottlenecks**
- **identify impacted users**



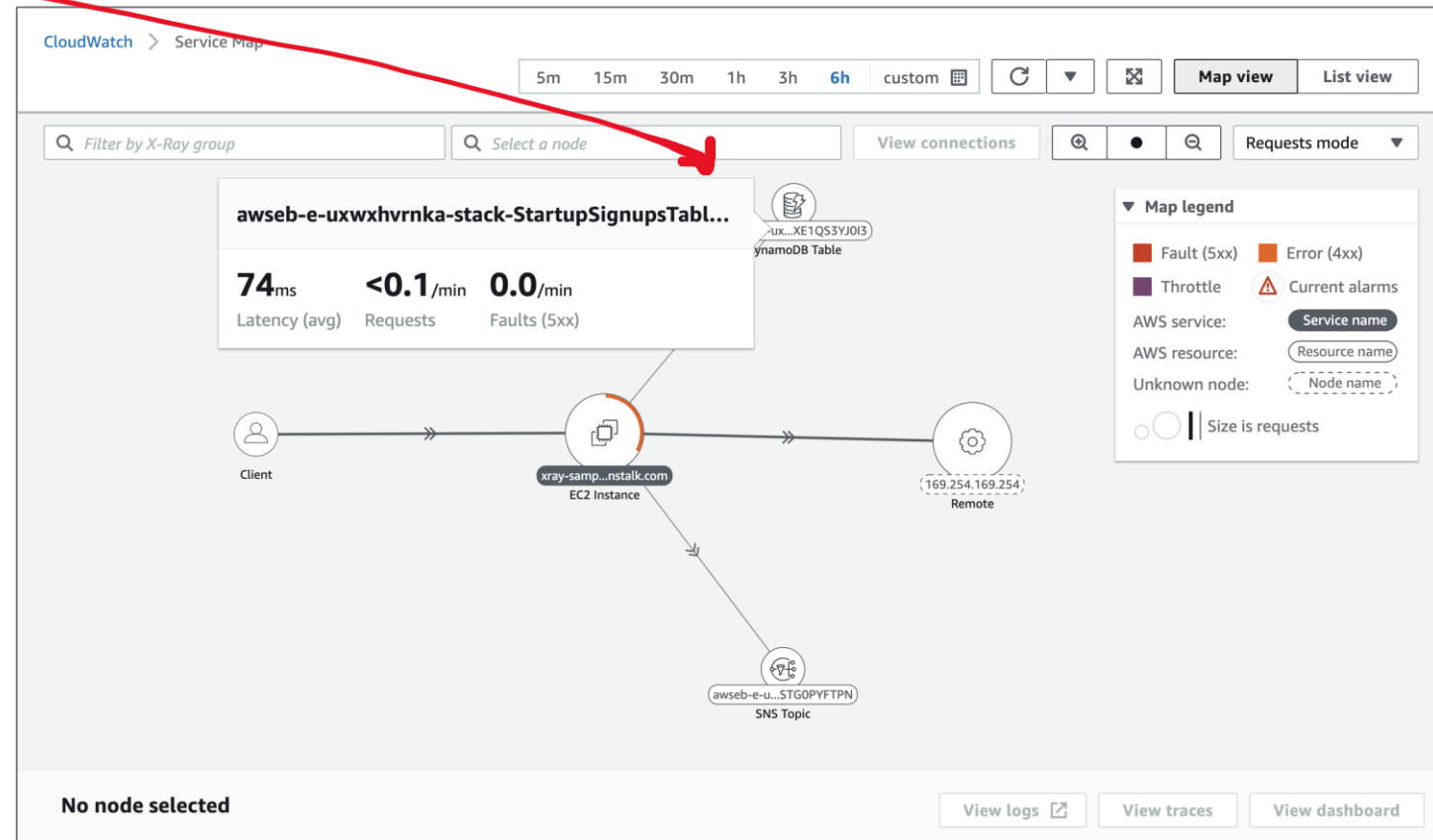
CloudWatch ServiceLens

Service Map displays your service endpoints as **“nodes”** and highlights the traffic, latency, and errors for each node and its connections

- ServiceLens integrates with **CloudWatch Synthetics**
- ServiceLens supports **log correlation** with:
 - Lambda Functions
 - API Gateway
 - Java-based apps on EC2
 - Java-based apps on ECS
 - Java-based apps on EKS
 - Kubernetes with Container Insights

To install and use Service Lens, you need to:

- Deploy X-Ray (Instrument your services)
- Deploy CloudWatch Agent and X-Ray daemon





CloudWatch ServiceLens

CloudWatch > Service List

5m 15m 30m 1h 3h 6h custom Map view **List view**

Filter by X-Ray group

Nodes (4) View on map View logs View traces View dashboard

Find nodes

	Name	Type	Alarms	Latency (avg)	Faults (5xx)	Requests
☐	169.254.169.254	Remote	-	2ms	0.00/min	0.01/min
☐	awseb-e-uxwxhvrnka-stack-NewSignupTopic-RUSTG0PYFTPN	SNS Topic	-	70ms	0.00/min	<0.01/min
☐	awseb-e-uxwxhvrnka-stack-StartupSignupsTable-16XE1QS3YJ0I3	DynamoDB Table	-	74ms	0.00/min	<0.01/min
☐	xray-sample.eba-mzgxi9i8.us-west-1.elasticbeanstalk.com	EC2 Instance	-	54ms	0.00/min	<0.01/min

Current alarms (0)

Find alarms

Alarm name	State	State updated	Related nodes
No alarms found			

ServiceLens has **two modes**:

- Map View
- List View

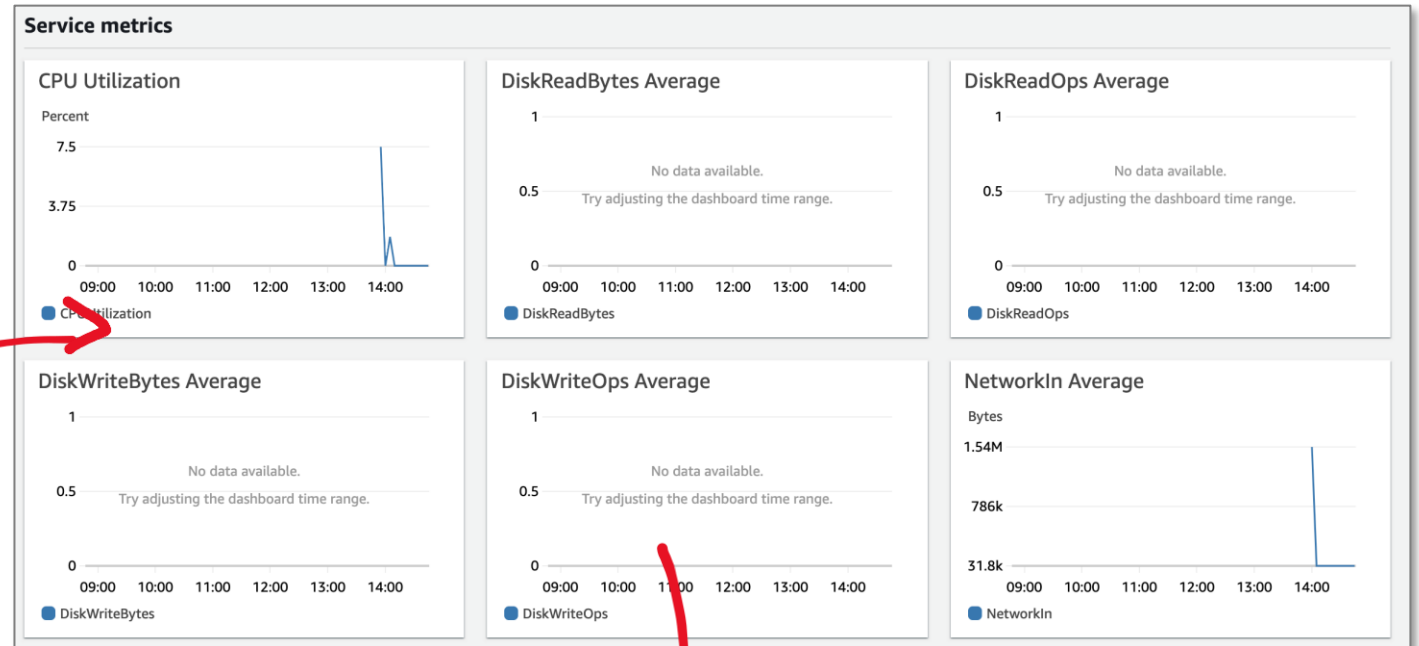
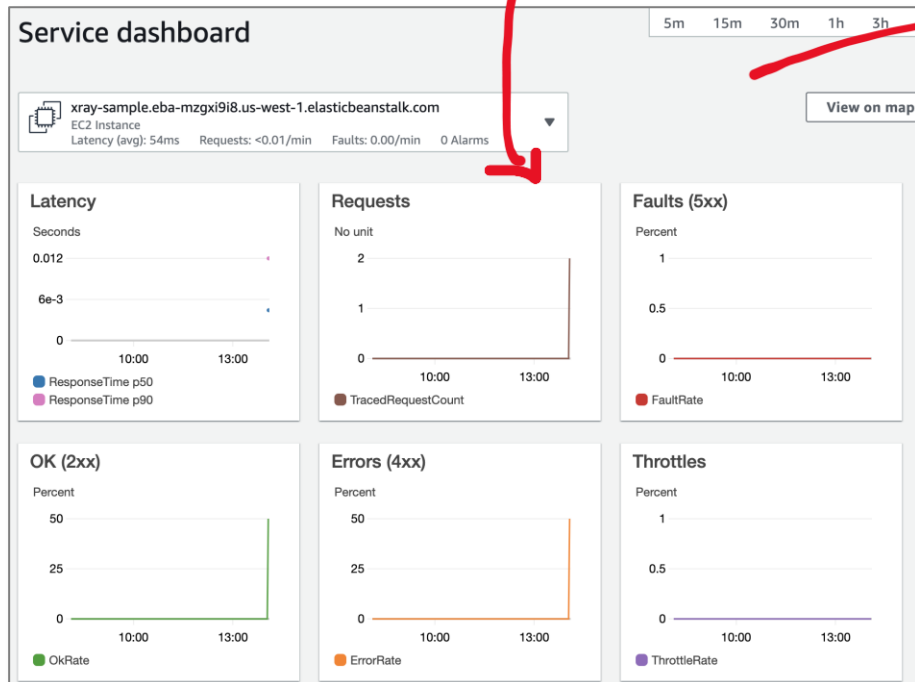
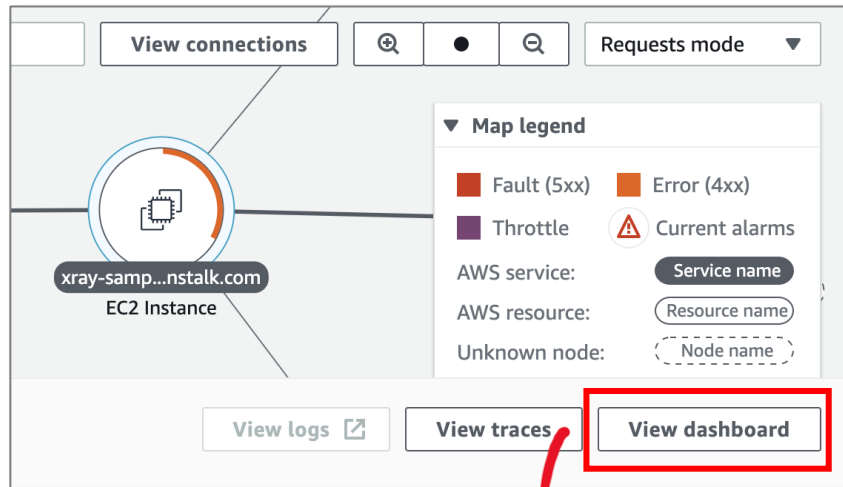
Map View is the Service Map showing us traces between modes

List View is a flat list of **Nodes** that make up the service.



CloudWatch ServiceLens

Clicking into a node, we get a **Service Dashboard** with lots of information



InstanceId	Name	InstanceType	State	Monitoring
i-08d0f29a0d500255e	xray-sample	t3.large	running	disabled

50



CloudWatch ServiceLens

ServiceLens lets us quickly **filter trace information** to open in **X-Ray Analytics**

Filters [Info](#)

Choose a set of filters for your traces. The filters are based on fields found in the retrieved traces. To find traces using other filters use the custom query.

Filter type

☒ Trace status

☐ Response time

☐ Custom query

☐ HTTP Method

☐ Response code

☐ URL

☐ Resource ARN

☐ User

☐ User agent

☐ Client IP

Filter traces by trace status

Select rows from the table and add them to your filter

☐	Trace status	Successful requests	% of retrieved traces	Mean response time
☐	Error	0.0%	40%	0.02s
☐	Throttle	0%	0%	0s
☐	Fault	0%	0%	0s
☐	OK	100.0%	60%	0.06s

Summary details of traces

Number of traces
5

Mean response time
40ms

Response time distribution of traces

Select range to filter by response time

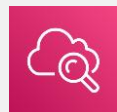
P50

P95 P99

Add selected response times to filter

Traces (5)

	ID	Trace status	Timestamp	Response code	Response Time	HTTP Method	URL Address
☐	1-5f15a517-b0bbfdacea b5113c89ad3124	OK	44.3min (2020-07-20 10:07:19)	200	0.008s	GET	http://54.219.18.10/
☐	1-5f15a47f-52b2d22af8c 10ba75bb592f5	Error	46.8min (2020-07-20 10:04:47)	404	0.004s	GET	http://xray-sample.eba-mzgxi9i8.us-west-1.elasticbeanstalk.com/favicon.ico
☐	1-5f15a518-41926776c7 8ef7037b429595	Error	44.3min (2020-07-20 10:07:20)	409	0.028s	POST	http://xray-sample.eba-mzgxi9i8.us-west-1.elasticbeanstalk.com/signup
☐	1-5f15a47e-fec50d963e 1a4b65d5938e59	OK	46.8min (2020-07-20 10:04:46)	200	0.012s	GET	http://xray-sample.eba-mzgxi9i8.us-west-1.elasticbeanstalk.com/
☐	1-5f15a4dc-138d612a14 c1b19adba7edd3	OK	45.3min (2020-07-20 10:06:20)	201	0.147s	POST	http://xray-sample.eba-mzgxi9i8.us-west-1.elasticbeanstalk.com/signup



CloudWatch Synthetics

Synthetics is used **to test web applications** by

creating **canaries** to:

- Broken or dead links
- Step-by-step task completion
- Page load errors
- Load latencies of assets
- Complex Wizard flows
- Checkout flows

What is a Canary?

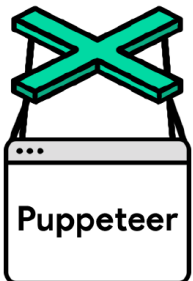
Canaries are **configurable scripts** that **run on a schedule**, to monitor your endpoints and APIs

Canaries mimic steps a real user would take so you can **continuously verify the customer experiercer**.

Canaries run on



AWS Lambdas using the **Node.js** and **Puppeteer**



Headless means that there is no visible window. So you don't "see" the browser.

Puppeteer is a **headless chrome browser** and an **automated testing framework**.

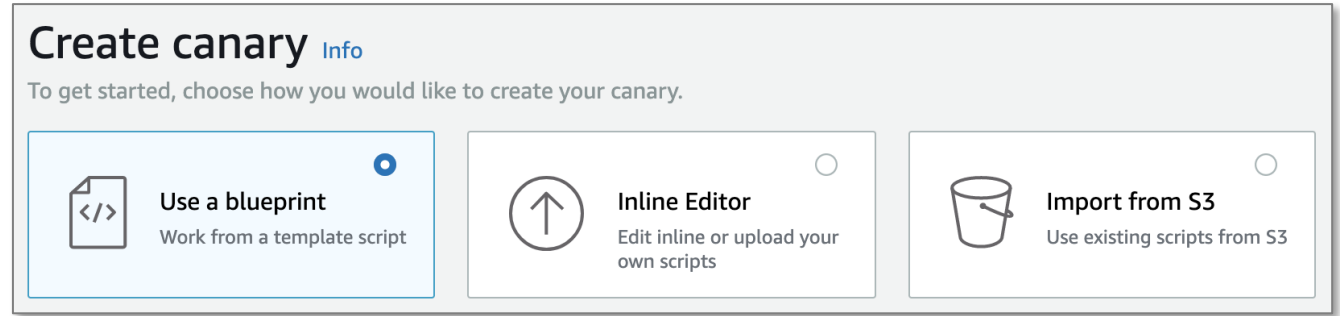
You can code Puppeteer to open a web-browser and click and enter information into a website.



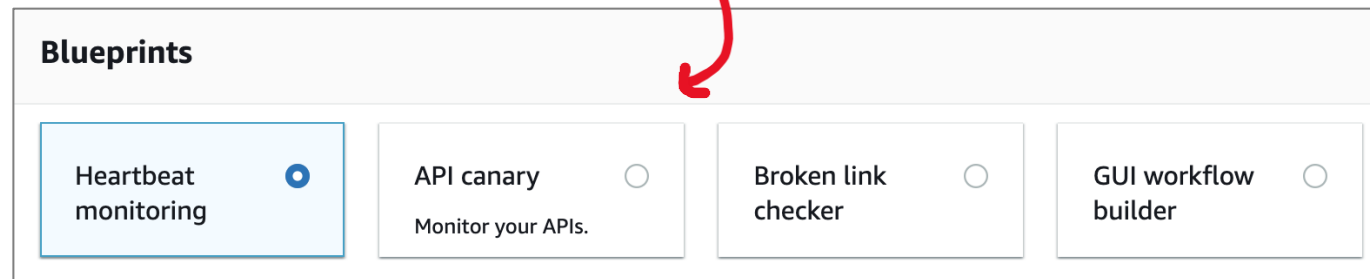
CloudWatch Synthetics

When creating a canary, you can:

- use a **blueprint**
- use the **inline-editor**
- **Import from S3**



There are 4 Blueprints to start with:





CloudWatch Synthetics – Heartbeat Monitoring

Heartbeat Monitoring

Used to check a single page

Supply a single URL

Wait a while, and then take a screenshot
When the page has loaded

It's called Heartbeat Monitoring because it
checks continuously to see if the page is
still live.

Application or endpoint URL [Info](#)

Enter the endpoint, API or url that you are testing.

Script editor

Undo

Clear editor

```
6 // INSERT URL here
7 const URL = "https://app.exampro.co/signup";
8
9 let page = await synthetics.getPage();
10 const response = await page.goto(URL, {waitUntil: 'domcontentloaded', timeout: 30000});
11 //Wait for page to render.
12 //Increase or decrease wait time based on endpoint being monitored.
13 await page.waitFor(15000);
14 await synthetics.takeScreenshot('loaded', 'loaded');
15 let pageTitle = await page.title();
16 log.info('Page title: ' + pageTitle);
17 if (response.status() !== 200) {
18     throw "Failed to load page!";
19 }
```

☒ Run continuously

Best option for long running monitoring projects. Must be in the range of 1-60 minutes, inclusive.

Run every minute(s)

☒ Start immediately after creation



CloudWatch Synthetics – API Canary

API Canary

Used to check an API endpoint

Supply the API endpoint

- Method (GET, POST)
- Headers
- Payload (data)

Method
The HTTP request the canary will be testing

GET

Application or endpoint URL [Info](#)

https://api.exampno.co/get-catalogue

Enter the endpoint, API or url that you are testing.

Headers

Key	Value	
X-Access-Key	123dfs802rdsf0s8	Remove header

Add header

Data

{ "cloud_service_provider": "aws" }

Checks if **200 is returned for success**, anything else is considered a failure.

Script editor Undo Clear editor ⌕

```
7   const postData = `{ "cloud_service_provider": "aws" }`;
8
9   const verifyRequest = async function (requestOption) {
10    return new Promise((resolve, reject) => {
11      log.info("Making request with options: " + JSON.stringify(requestOption));
12      let req
13      if (requestOption.port === 443) {
14        req = https.request(requestOption);
15      } else {
16        req = http.request(requestOption);
17      }
18      req.on('response', (res) => {
19        log.info(`Status Code: ${res.statusCode}`)
20        log.info(`Response Headers: ${JSON.stringify(res.headers)}`)
21        if (res.statusCode !== 200) {
22          reject("Failed: " + requestOption.path);
23        }
24        res.on('data', (d) => {
25          log.info("Response: " + d);
26        });
27      });
28      res.on('end', () => {
```



CloudWatch Synthetics – Broken Link Checker

Broken Link Checker

Supply link(s), then look for links on the page and follow them to see if any of those links are broken

Tell me what website it should look at

How many links on the page should it click on and see if they load

You can supply multiple URLs

It will **log** all the pages it was able to load or not load.
Since Canaries use AWS Lambda, it would just log to
a CloudWatch Log Group.

Application or endpoint URL [Info](#)

Enter the endpoint, API or url that you are testing.

Maximum number of links which would be followed

Script editor

[Undo](#)

[Clear editor](#)



```
18     });
19     return urls;
20 }
21
22 //Get the filename for the screenshot based on the URI
23 const getFilename = function(url, defaultName = 'loaded') {
24
25     if (!url) return defaultName;
26
27     const uri      = new URL(url);
28     const pathname = uri.pathname.replace(/\/$/, ''); //remove trailing '/'
29     const filename = !!pathname ? pathname.split('/').pop() : 'index';
30
31     //Remove characters which can't be used in S3
32     return filename.replace(/^[^a-zA-Z0-9-_.!*'()]+/g, '');
33 }
34
35 const webCrawlerBlueprint = async function () {
36     // INSERT URL here
37     let urls = ["https://www.exampco.co"];
38
39 }
```



CloudWatch Synthetics – GUI Workflow Builder

GUI Workflow Builder

Test a sequence of steps that makes up a workflow

You add actions such as Click, Input Text, and Verify Text.

Application or endpoint URL [Info](#)
Enter the endpoint, API or url that you are testing.

Workflow builder
Select the actions you would like the canary to take.

Action	Selector	Text	
Input text	[id='email']	new@user.com	Remove action
Input text	[id='full_name']	New User	Remove action
Input text	[id='password']	SecurePassword1!	Remove action
Click	[id='submit']		Remove action
Add action			

Action

- ▲
Type text into the target element
- Click
Click on an element in the page
- Verify selector
Verify that an element exists
- Verify text
Verify a string within the target element
- Type text into the target element
- Click with navigation
Click on an element and expect navigation

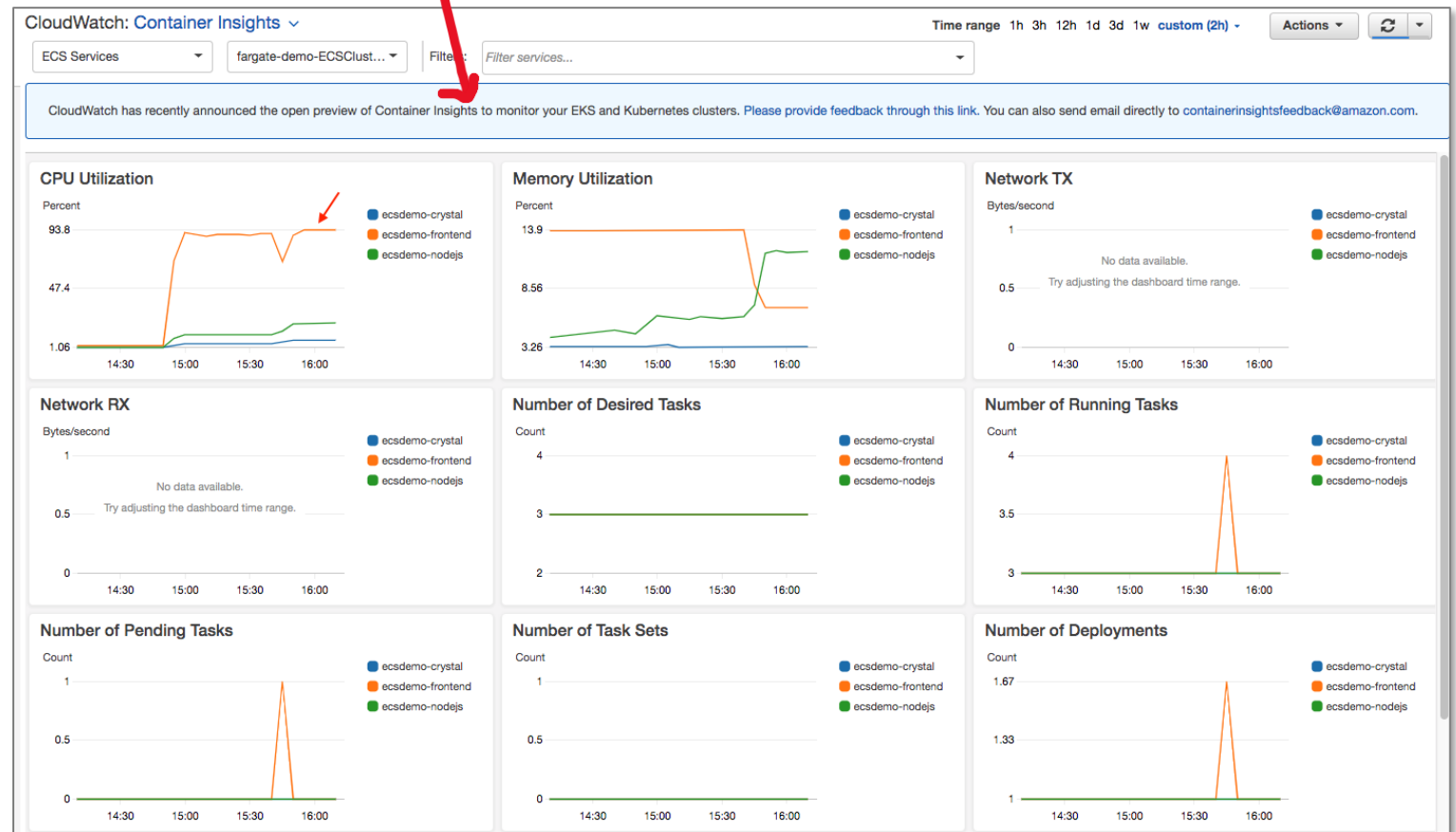


CloudWatch Container Insights

Container Insights **collects**, **aggregates** and **summarizes** information about your containers from **metrics and logs**

Container Insights works with:

- Elastic Container Service (ECS)
- ECS Fargate
- Elastic Kubernetes Service (EKS)
- Kubernetes running on EC2 instances



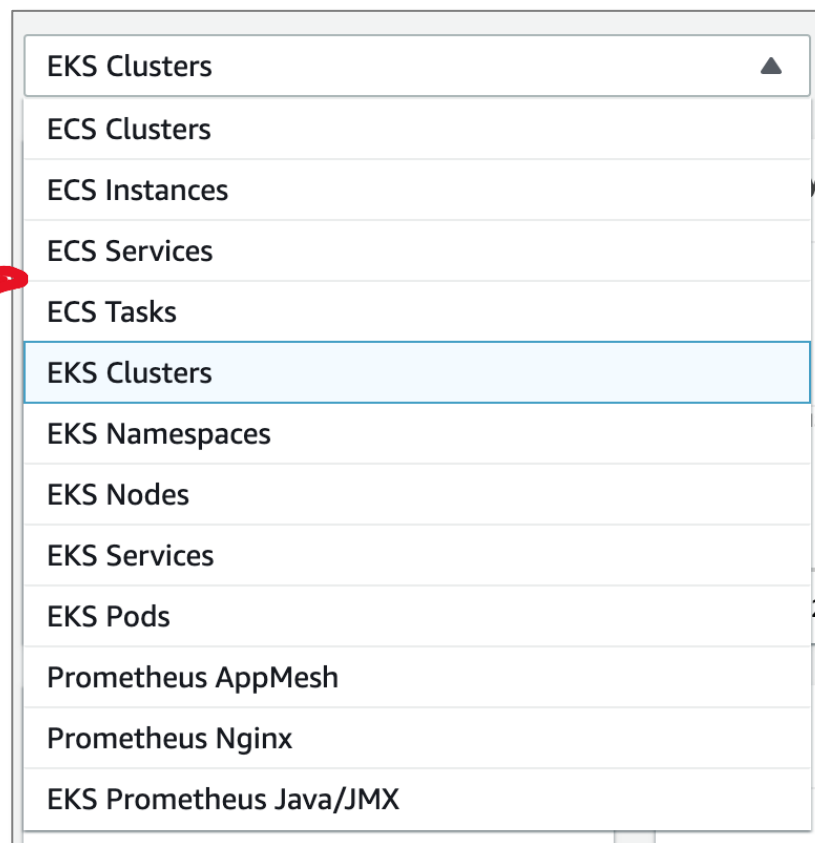


CloudWatch Container Insights

- Metrics that Container Insights collects are **available in CloudWatch automatic dashboards**
- You can **analyze and troubleshoot** container performance and logs data with **CloudWatch Logs Insights**
- Operational data is collected as **performance log events**.
 - These entries use a structured JSON schema that enables high-cardinality data to be ingested and stored at scale.

Container Insights can be **filtered** by

- Cluster
- Node
- Pod
- Task
- Service Level





CloudWatch Contributor Insights

Contributor Insights allows you to **view top contributors** impacting the performance of your systems and application in **real-time**.

Contributor Insights looks at **your CloudWatch Logs**, and based on **Insight rules** you define, shows **real-time time-series data**.

AWS has a bunch of **sample rules** you can use to get started:



☒ Sample rule Select sample rule ▾

Select log group(s)

► Select by preference

- ☒ JSON
- ☐ CLF (Common Log Format)

\$. Unique key

API Gateway

Route 53 Query Logs

Container Insights Logs - Amazon EKS & Kubernetes

Container Insights Logs - Amazon ECS

VPC Flow Logs

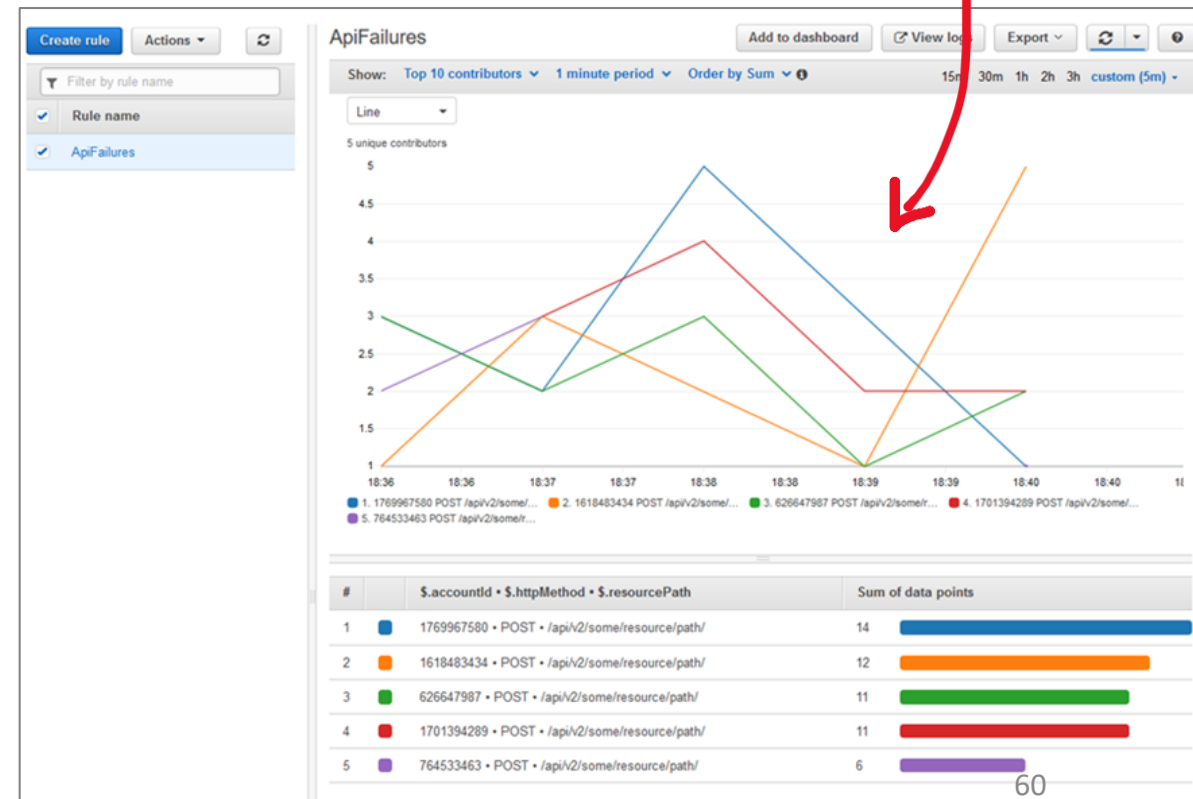
Byte Transfers by Source and Destination IP Address

Top HTTP Packets By Destination Address

Top Sources by Rejected TCP Connections

Traffic by Source Address and Action

Traffic by Source Address and Rejected Action





CloudWatch *CheatSheet*

- The Pillars of Observability are **Metrics**, **Logs** and **Traces**. You need to use all three together to obtain **Observability**
- AWS CloudWatch is a collection of monitoring services:
 - **CloudWatch Logs** — centralization log management service. Logs from AWS Services, Application Logs running on your EC2s
 - **CloudWatch Log Insights** — Robust way to filter your logs, it auto-detects log patterns creates dynamic fields, compose queries.
 - **CloudWatch Metrics** — Aggregates datapoints from AWS Service to produce a metric. eg AvgCPUUtilization
 - **CloudWatch Alarms** — Define a threshold up on a metric and reacts by triggering action such as EC2, Auto Scaling or SNS action
 - **CloudWatch Events (EventBridge)** — A serverless account wide event bus. Create event-driven loosely coupled apps with AWS Services
 - **CloudWatch Dashboards** — Create dashboards filled with graphs powered by CloudWatch metrics
 - **CloudWatch Container Insights** — collect, aggregate, and summarize metrics and logs from your containerized apps and microservices
 - **CloudWatch Contributor Insights** — view the top contributors impacting the performance of your systems and applications in real-time
 - **CloudWatch Synthetics** — Test your web-apps to see if they're broken. Uses puppeteer underneath.
 - **CloudWatch Service Lens** — "Observability" for distributed or serverless apps by pulling X-Ray Traces, CloudWatch Logs, and Metrics



CloudWatch Logs *CheatSheet*

CloudWatch Logs is used to **monitor**, **store**, and **access** your log files

- Export logs to S3 to analyze with Athena
- Stream logs to ElasticSearch Service to run full-text search
- Stream CloudTrail Events to CloudWatch Logs to allow you to react to CloudTrail Events
- Encrypted by default, You can apply your own KMS Key
- Retains logs indefinitely by default you can choose a custom retention period between 1 Day and 10 years
- Support a simple Filtering Syntax
- **Log Groups** — A container for collection of log streams. Uses the forward slash naming convention eg. /my-app/prod/us-east/
- **Log Streams** — Represent a log, usually created by AWS Services, can be manually created
- **Log Events** — Represents a single event (a single line from the log file)
- In order to send custom application logs running on your EC2 instance you need to install the **SSM CloudWatch Agent**
- When the agent is installed, you update the **/etc/awslogs/awslogs.conf** on your EC2 instance to include the logs you want to send to CloudWatch



CloudWatch Logs Insights *CheatSheet*

CloudWatch Logs Insights enables you to **interactively search and analyze your CloudWatch log data**

- CloudWatch Logs Insights supports all kind of logs
- CloudWatch Logs Insights is generally used from the AWS Console to create ad-hoc queries
- CloudWatch Logs Insights has its own language called CloudWatch Logs Insights **Query Syntax**
- A single request can query up to **50 log groups**.
- Queries **time out after 15 minutes**, if they have not completed.
- Query results are **available for 7 days**.
- CloudWatch Logs Insights comes with a bunch of sample queries
- CloudWatch Logs Insights analyzes the log events in imported logs to generate out **Dynamic Fields**
- CloudWatch Logs Insights always generates out 5 of the following fields:
 - **@message** — the raw unparsed log event.
 - **@timestamp** — the event timestamp contained in the log event's timestamp field.
 - **@ingestionTime** — the time when the log event was received by CloudWatch Logs.
 - **@logStream** — the name of the log stream that the log event was added to.
 - **@log** — is a log group identifier in the form of account-id:log-group-name.
- CloudWatch auto-discovery fields for the following AWS Services
 - Amazon VPC Flow Logs
 - Amazon Route53
 - AWS Lambda
 - AWS CloudTrail
- JSON logs keys will all become discovered fields
- For other logs that are not automatically discovered you can use the parse command to create ephemeral fields



CloudWatch Metrics *CheatSheet*

CloudWatch Metrics represents a **time-ordered set of data points** Its a **variable** that is **monitored over time**.

- Metrics are data about the performance of your systems.
- CloudWatch comes with many **predefined** metrics that are generally name spaced by AWS Service.
- Very common predefined metrics at the EC2 Per-Instance Metrics: CPUUtilization, DiskReadOps, DiskWriteOps, DiskReadBytes, DiskWriteBytes, NetworkIn, NetworkOut, NetworkPacketsIn, NetworkPacketsOut
- Metric data is kept for 15 months, enabling you to view both up-to-the-minute data and historical data.
- **Metric math** enables you to **query multiple CloudWatch metrics** and **use math expressions** to create new time series
- You can publish your own metrics using the AWS CLI or AWS SDK eg. `aws cloudwatch put-metric-data`
- You can define custom metrics in two different resolutions:
 - standard resolution (1 minute)
 - high resolution (> 1 minute to 1 second)
- With High Resolution you can track in intervals of: 1 second, 5 seconds, 10 seconds, 30 seconds, multiple of 60 seconds.
- Data availability (when you can see the data) varies based on service
 - **EC2 Basic Monitoring (5 minutes)**
 - EC2 Detailed Monitoring (1 minute)
 - Other Services generally 1 minutes but could also be 3 or 5 minutes
- You can turn on EC2 Detailed Monitoring for a price
- Not all EC2 metrics are tracked by default you have to install the CloudWatch Agent to collect
 - **Memory utilization** (Am I running out of memory?)
 - Disk Swap utilization
 - **Disk Space utilization** (Am I running out of disk space?)
 - Page file utilization



CloudWatch Alarms *CheatSheet*

A **CloudWatch Alarm** monitors a **CloudWatch Metric** based on a **defined threshold**.
When an alarm **breaches** (goes outside the defined threshold) than it changes **state**.

Metric Alarm States

- **OK** The metric or expression is **within** the defined threshold
- **ALARM** The metric or expression is **outside** of the defined threshold
- **INSUFFICIENT_DATA**
 - The alarm has **just started**
 - the metric is **not available**
 - **Not enough data** is available

When it changes state, we can define what **action it should trigger**:

- Notification
- Auto Scaling Group
- EC2 Action
- You can define Conditions of either a **Static** or **Anomaly Detection**
 - **Static** set a static value as the threshold eg. 100 USD
 - **Anomaly Detection** sets a band around the data points, helps prevents false-positives, more flexible if you have seasonal data
- **Composite Alarms** — Allows you watch multiple alarms and require both to trigger before resulting in an alarm action
 - The alarms being watch must have no actions sets
 - You can only trigger an SNS as the action (so no EC2 or ASG actions)



EventBridge *CheatSheet*

EventBridge is a serverless **account-wide** or **organization** event bus. Create event-driven loosely coupled apps with AWS Services

- EventBridge was formerly known as CloudWatch Events (both are still accessible within the AWS console)
- **An event bus receives events** from a **source** and **routes events** to a **target** based on **rules**
- EventBridge/CloudWatch **Events** are JSON objects that are used to pass data around
- Many AWS Services emit Event data, but for AWS services that do not, you can turn on CloudTrail and react to those events.
- AWS API call events that are **larger than 256 KB** in size are not supported.
- To react to event data, you need to create an **EventBridge Rule**
 - You can choose up to 5 AWS Service Targets (eg. Lambda Function, SQS queue, SNS topic, Firehose delivery stream, ESC Task)
 - When you select a target, you have additional fields to narrow down the AWS Service you want to target
 - You can apply Event Matching to filter what events should be passed to the Target
 - You simply provide a JSON schema with the fields you want to match on, or you can use complex matches:
 - **Prefix Matching** match on the prefix of a value in the event source
 - **Anything-but Matching** matches anything except what's provided in the rule
 - **Numeric Matching** matches against numeric operator for "<", ">", "=", "<=", ">="
 - **IP Address Matching** matches against available IP addresses for both IPv4 and IPv6
 - **Exists Matching** matches works on the presence or absence of a field in the JSON
 - **Empty Value Matching** For strings, you can use "" to match empty. For other values, you can null
 - **Multiple Matching** combines multiple matching rules into a more complex event pattern
 - You can set **Configure Input** which is used to transform / filter the **event's data structure** that will be passed:
 - **Match Event** The entire event pattern text is passed to the target when the rule is triggered.
 - **Part of the matched event** only the part of the event text that you specify is passed to the target.
 - **Constant (JSON text)** send static content instead of the matched event data. (Mocked JSON)
 - **Input Transformer** You can transform the event text into a different format of a string or a JSON object



EventBridge *CheatSheet*

- A common use case of EventBridge is to use it as a serverless cron job. (eg. Trigger database backup every day)
- EventBridge can schedule events using either **Cron Expression** or **Rate Expressions**
 - **Cron Expression** – very fine tune control (eg. 15 12 * MON-FRI * = 12pm UTC Monday to Friday)
 - **Rate Expression** — Easy to set, not as fine-grained (eg. Choose every X Hours / Minutes / Days)
 - All scheduled events use **UTC time zone**
 - the minimum precision for schedules is **1 minute**.
- The event bus can extend to third-party SaaS Products to accept **Partner Event Sources** (eg. React to Datadog event, and trigger ASG)
- **EventBridge Schema Registry** allows you to keep track of changes to your event schemas an:
 - It will automatically detect the schema changes and create versions
 - You can download the Schema as Code Bindings
 - You can use the AWS Toolkit VSCode plugin to view generated Schemas or apply Code Bindings